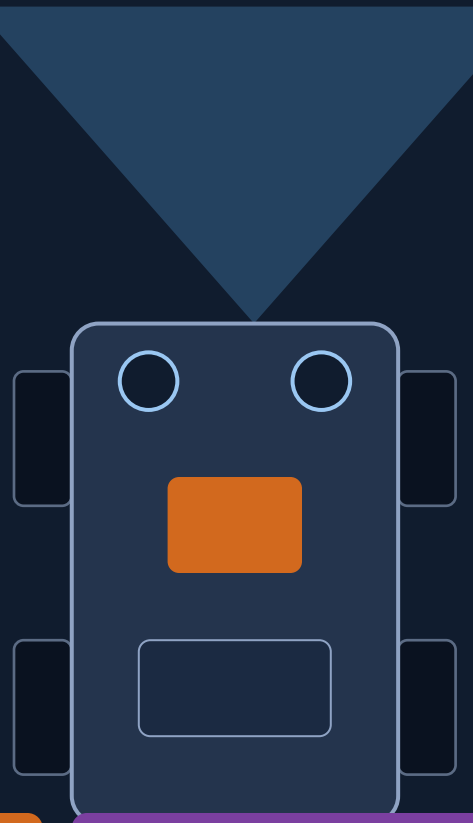
자율주행 미니카 실습 교안

4륜 RC카 + ESP32-CAM 영상 모듈 · 1팀부터 4팀까지 통합본

미래모빌리티공학과 2학년 2학기



1

라인트레이싱
+ 장애물 정지

2

블루투스
수동/자동 전환

3

영상 수신
+ 색상 인식

4

영상 인식 → 정지
명령 → 주행

목차

어디서부터 볼까요

처음에 **이 자료 보는 법** **네 팀의 연결**

공통 기초

장비 한눈에
전원과 안전
핀 약속
PWM, 모터 드라이버
조향 원리
값 확인, millis
교재 코드 대응표

1팀 라인트레이싱

시작 전
센서 원리
STEP 1~2 센서, 임계값
STEP 3~4 초음파, 모터
STEP 5 라인트레이싱
STEP 6~7 장애물, 측정
심화 A~C

2팀 블루투스

시작 전 (교재 부록)
블루투스 원리, 배선
STEP 1~3 에코, 명령, 주행
STEP 4 앱
STEP 5 모드 전환
STEP 6 안전장치
심화

3팀 영상 인식

시작 전
파이썬 기초 1~3
영상 경로와 Wi-Fi
STEP 1~3 연결, 설치, 영상
HSV 색공간
STEP 4~7 색 검출, 컨투어
STEP 8~9 대시보드, 측정

4팀 인식에서 주행까지

전체 흐름
시작 전 점검
STEP 1~2 PC 블루투스
STEP 3 차량 수신
STEP 4~5 판단, 통합
STEP 6 지연시간
STEP 7 시험

부록

용어 사전
증상별 찾아가기
추천 링크
운영 규칙
기록 양식

처음에

이 자료 보는 법

1 네 팀의 교안이 한 파일에 들어 있습니다. 내 팀 순서에 매이지 말고 필요한 곳으로 가서 보세요.

2 모든 슬라이드 맨 아래의 **목차 · 공통 · 1팀 ~ 4팀 · 부록** 링크로 이동합니다. 슬라이드 쇼에서는 클릭, 편집 화면에서는 Ctrl+클릭입니다.

3 STEP 슬라이드 위쪽의 **'먼저 되어 있어야 하는 것'** 링크는 앞 단계로 바로 갑니다. 오른쪽 위 동그라미 번호도 눌러서 이동할 수 있습니다.

4 코드의 핀 번호는 예시입니다. 내 차량 배선과 다르면 바꿔서 쓰고, 바꾼 번호를 노트에 적어 둡니다.

5 각 STEP 끝의 **확인** 항목이 모두 되면 다음 단계로 넘어갑니다.

6 1팀이 먼저 끝났다면 2팀 주제로, 3팀이 끝났다면 4팀 주제로 이어서 진행합니다. 각 팀 마지막 장에 방법이 있습니다.

STEP 슬라이드는 이렇게 생겼습니다

항목	내용
목표	이 STEP이 끝나면 되는 것
할 일	순서대로 따라 하기
코드	복사해서 쓰되 의미를 읽고 쓸 것
확인	다음으로 넘어가도 되는 기준
막히면	문제 해결 슬라이드나 앞 단계

참고 표시된 숫자(속도, 거리, 면적 기준값)는 출발점입니다. 차량과 교실 조명에 따라 달라지므로 직접 측정해서 바꿉니다.

처음에

교재 200쪽 이후, 이 교안이 이어지는 방식

이미 한 것 (교재 ~200쪽)

- 서보모터, 초음파 센서, 라인트레이서 값 읽기
- 모터 드라이버(L298N)와 4륜 주행
- 시리얼 제어, 적외선 리모컨 제어
- 라인트레이서 주행(9번), 장애물 감지(10번)

아직 하지 않은 것

- 블루투스 (교재 부록, 203쪽부터)
- 파이썬, OpenCV, ESP32-CAM 영상 수신 (교재에 없음)

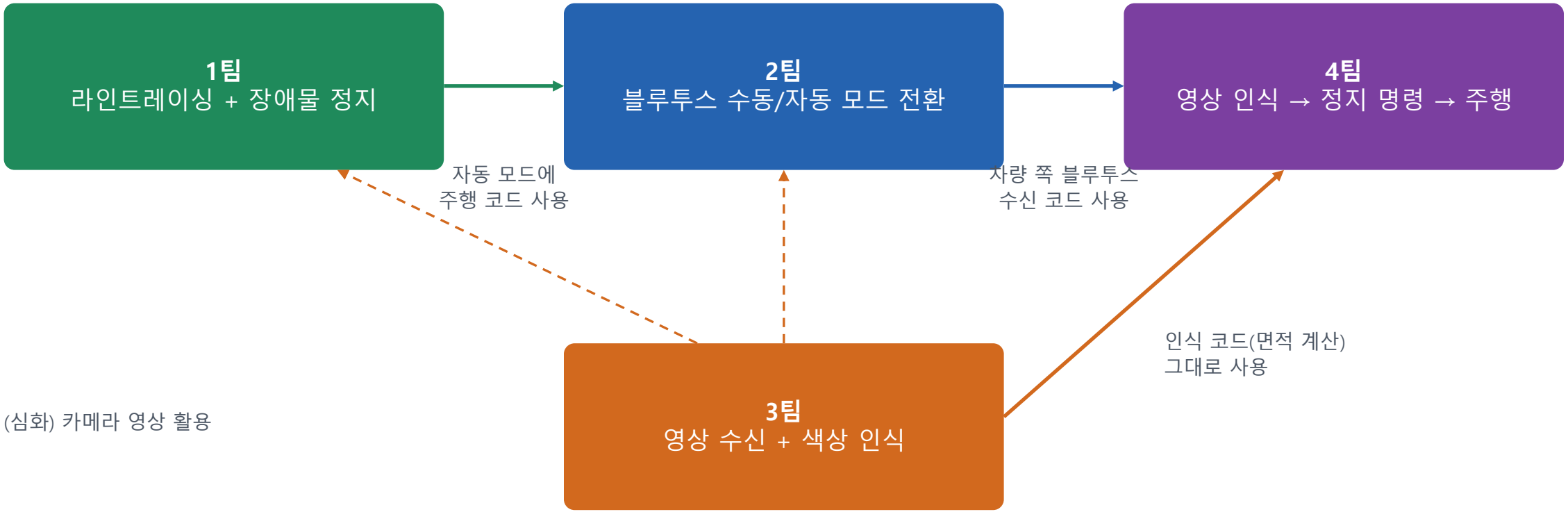
팀	먼저 볼 교재	프로젝트에서 새로 하는 것
1팀	라인 센서 51~61쪽 라인 주행 182~190쪽 장애물 192~200쪽	두 예제를 하나로 합치기, millis, 임계값, 히스테리시스, 안정성 측정
2팀	부록 203~242쪽 (블루투스, 앱)	수동/자동 모드 전환, 끊김 안전장치
3팀	교재에 없음	파이썬, OpenCV, ESP32-CAM 영상 수신, 색 검출
4팀	1팀, 2팀, 3팀 결과	인식에서 판단, 전송, 정지까지 연결

참고 쪽수는 첨부한 교재(4휠 RC카 강의자료 v4_1) 기준입니다. 각 팀 시작 장과 STEP 제목 옆의 '교재 ...' 표시가 같은 쪽수를 알려 줍니다.

처음에

네 팀은 서로 이어져 있습니다

1팀 주행이 안 되면 2팀의 자동 모드가 안 되고, 3팀 인식이 안 되면 4팀의 정지 판단이 안 됩니다. 막히면 앞 팀 슬라이드로 돌아가세요.



실선: 꼭 필요한 연결 점선: 심화 과제에서만 쓰는 연결

공통

공통 기초

어느 팀이든 한 번은 읽어 두는 부분입니다. 전원, 핀, PWM, 모터 드라이버, 조향 원리, 값을 눈으로 확인하는 방법을 모았습니다.

차례

01 장비 한눈에 보기

02 전원과 안전

03 핀 약속과 배선 확인

04 PWM

05 모터 드라이버와 drive 함수

06 조향 원리

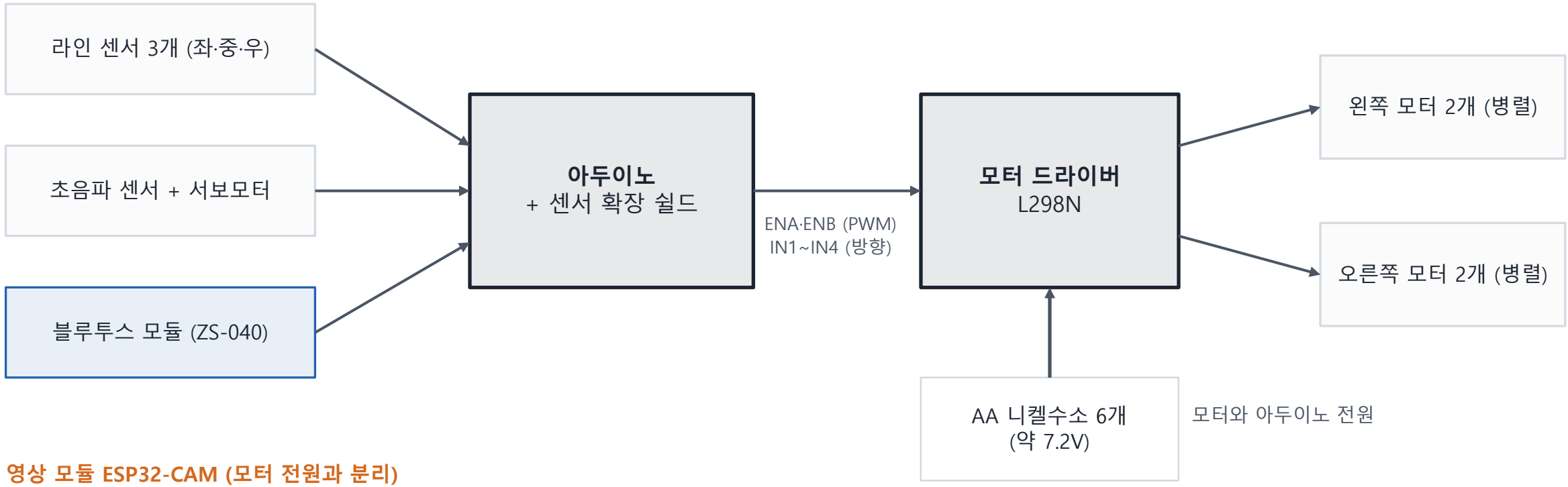
07 값부터 눈으로 확인하기

08 delay 대신 millis

09 교재 코드와 교안 코드 대응표

장비 한눈에 보기

차량 안에서 일어나는 일과 영상 모듈(ESP32-CAM)을 한 장에 그렸습니다. 3팀과 4팀은 ESP32-CAM을, 2팀과 4팀은 블루투스 모듈을 추가로 씁니다.



영상 모듈 ESP32-CAM (모터 전원과 분리)

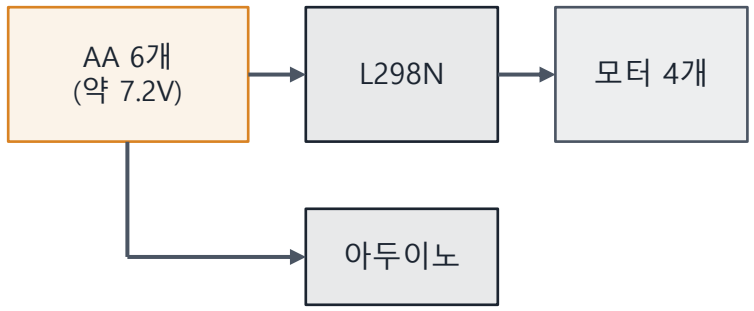


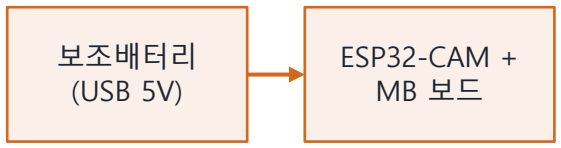
차량 위 — 책상 위

전원과 안전

대상	전원	메모
모터 + 아두이노	AA 니켈수소 충전지 6개 (약 7.2V)	충전지는 1.2V씩이라 6개가 7.2V입니다. 새 알카라인 6개(9V)와 다릅니다.
ESP32-CAM (차량)	보조배터리 USB 5V (MB 보드에 연결)	모터 전원과 따로 씁니다. 같이 쓰면 전압이 흔들려 영상이 끊기거나 보드가 재부팅됩니다.
ESP32-CAM 업로드	PC USB (MB 보드)	코드 업로드와 시리얼 모니터 확인은 MB 보드의 USB 단자로 합니다.

전원 연결





서로 연결하지 않습니다

지켜야 할 것

- **보조배터리**는 1A 이상 출력을 씁니다. 쓰는 전류가 작으면 저절로 꺼지는 제품도 있으니 미리 켜 두고 시험합니다.
- 배터리가 부풀거나 뜨거워지면 쓰지 않고 담당 선생님께 알립니다.
- +선과 -선이 닿지 않게 합니다. 단락은 발열과 화재로 이어집니다.
- **니켈수소**는 6개를 한 묶음으로 충전하고 교체합니다. 새 것과 헌것을 섞지 않습니다.
- 배선을 바꿀 때는 전원부터 끕니다.
- 카메라 리본 케이블은 **전원을 끈 상태에서만** 꽂고 뺍니다. 커넥터가 약해서 비스듬히 당기면 부러집니다.

알아 둘 것

L298N은 입력 전압에서 약 2V를 잃습니다. 7.2V를 넣으면 모터에는 5V 안팎이 갑니다. 배터리가 약해지면 낮은 PWM에서 바퀴가 아예 안 도는 일이 생깁니다.

공통 기초 교재 결선 131~140쪽

핀 약속과 배선 확인

에듀이노 4휠 스마트 RC카 강의자료(v4_1)의 결선표와 같은 번호입니다. 표의 번호는 센서 확장 쉴드에서 신호선(S)이 연결되는 핀입니다.

부품	연결 핀 (센서 확장 쉴드 S 핀)	코드에서의 이름
L298N ENA (오른쪽 속도, PWM)	5	ENA
L298N IN1, IN2 (오른쪽 방향)	8, 9	IN1, IN2
L298N ENB (왼쪽 속도, PWM)	6	ENB
L298N IN3, IN4 (왼쪽 방향)	10, 11	IN3, IN4
라인 센서 왼쪽 / 가운데 / 오른쪽	A5 / A4 / A3	PIN_L / PIN_C / PIN_R
초음파 센서 TRIG / ECHO	13 / 12	TRIG / ECHO
서보모터 (초음파 센서를 얹음)	2	myservo.attach(2)
적외선 수신 센서	A0	(리모컨 실습용)
블루투스 HC-06 (SoftwareSerial)	RX 3, TX 4	BT(3, 4)

배선 확인 요령

- 센서 확장 쉴드의 3핀 소켓은 **G(GND), V(VCC), S(신호)** 순서입니다. 센서의 GND, VCC, 신호선을 같은 순서로 꽂습니다.
- analogWrite는 PWM 핀에서만 됩니다. UNO 계열은 3, 5, 6, 9, 10, 11번(~ 표시)이고, 모터 속도 핀 5, 6번이 여기에 해당합니다.
- 키트 배선과 다르게 연결했다면 바꾼 핀 번호를 팀 노트 첫 장에 적어 둡니다.

공통 기초

교재 142~154쪽 복습

PWM: 켜고 끄는 비율로 속도를 정한다

analogWrite(pin, 64)

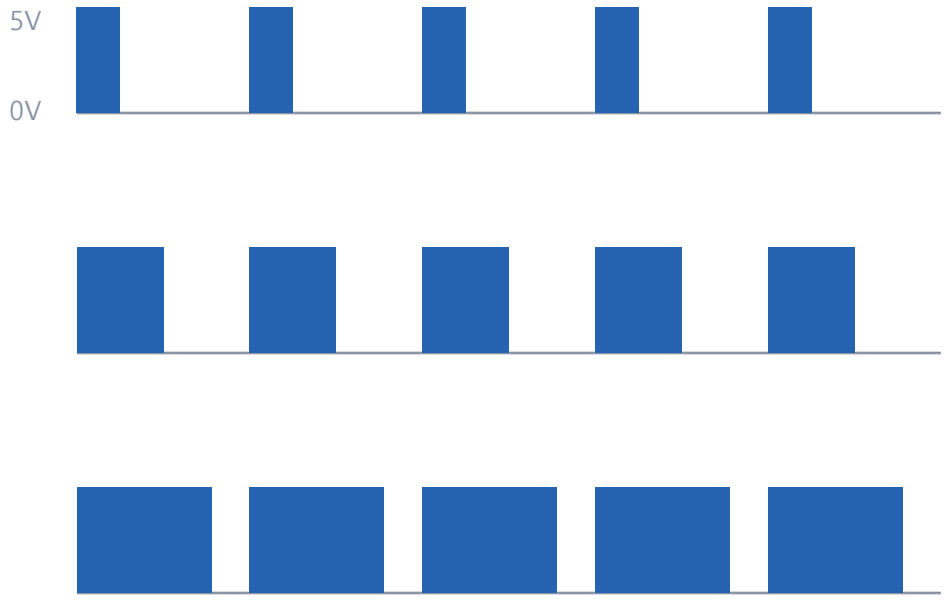
듀티 25%

analogWrite(pin, 128)

듀티 50%

analogWrite(pin, 200)

듀티 약 78%



한 주기 약 2ms (490Hz 기준)

- 아두이노 핀은 5V 아니면 0V, 두 가지만 낼 수 있습니다.
- analogWrite(핀, 값)은 0~255 값에 맞춰 핀을 아주 빠르게(약 490Hz, 모터 속도 핀인 5·6번은 약 980Hz) 켰다 껐다 합니다. 켜져 있는 비율이 값입니다.
- 모터는 반응이 느려서 평균 전력만큼의 속도로 돕니다. 128이면 대략 반쯤의 힘입니다.
- 값이 너무 작으면 모터가 아예 안 돕니다. 차량마다 **최소 구동값**이 다르므로 직접 찾습니다(1팀 STEP 4).
- analogWrite는 아날로그 입력 핀(A0 등)이나 analogRead와는 상관이 없습니다.

공통 기초 교재 6_Motor_driver 145쪽~

모터 드라이버(L298N)와 drive 함수

이 키트는 왼쪽 바퀴 2개가 한 채널, 오른쪽 바퀴 2개가 다른 채널에 병렬로 연결되어 있습니다. 앞뒤 바퀴를 따로 제어하지 않습니다. 교재는 IN1·IN2·ENA를 오른쪽 모터, IN3·IN4·ENB를 왼쪽 모터라고 부릅니다.

IN1	IN2	오른쪽 바퀴
HIGH	LOW	앞으로
LOW	HIGH	뒤로
LOW	LOW	멈춤

ENA에 주는 PWM 값이 오른쪽 속도입니다.
왼쪽은 IN3, IN4, ENB가 같은 역할을 합니다.

주의 바퀴가 반대로 돌면 그쪽 IN 두 선을 서로 바꿔 꽂거나, 코드에서 그쪽 부호를 뒤집습니다.

코드 공통 1 모터 함수 (모든 팀이 이 두 함수를 씁니다)

```
const int ENA = 5, IN1 = 8, IN2 = 9;     // 오른쪽 모터 2개
const int ENB = 6, IN3 = 10, IN4 = 11;   // 왼쪽 모터 2개

void setMotor(int en, int a, int b, int speed) {
  // speed: -255 ~ 255 (음수는 후진)
  digitalWrite(a, speed >= 0 ? HIGH : LOW);
  digitalWrite(b, speed >= 0 ? LOW : HIGH);
  analogWrite(en, constrain(abs(speed), 0, 255));
}

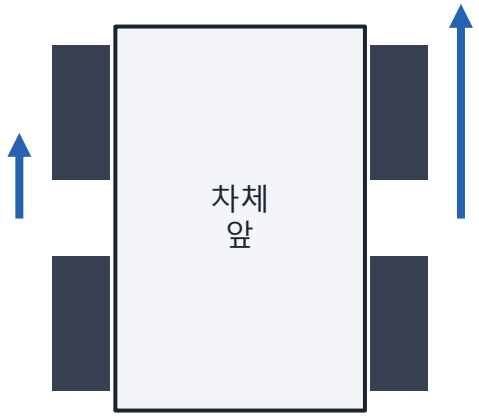
void drive(int left, int right) {
  setMotor(ENB, IN3, IN4, left);
  setMotor(ENA, IN1, IN2, right);
}

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
}
```

조향 원리: 좌우 속도 차이로 방향을 바꾼다

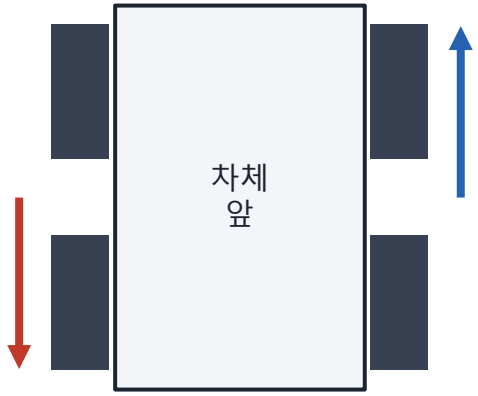
핸들이 없습니다. 왼쪽 바퀴들과 오른쪽 바퀴들의 속도를 다르게 주면 느린 쪽으로 차가 돕니다.

곡선 주행: drive(60, 150)



왼쪽 60 오른쪽 150

제자리 회전: drive(-120, 120)



왼쪽 -120 오른쪽 120

명령	결과
`drive(150, 150)`	직진
`drive(60, 150)`	왼쪽으로 휘며 전진
`drive(150, 60)`	오른쪽으로 휘며 전진
`drive(-120, 120)`	제자리 왼쪽 회전
`drive(0, 0)`	정지

용어 자동차의 '디퍼렌셜 기어(차동 장치)'와 다른 말입니다. 여기서는 좌우 바퀴의 속도 차이를 뜻합니다.

바퀴가 바닥에 끌리며 도는 방식이라 제자리 회전에는 평소보다 큰 PWM이 필요합니다.

공통 기초 교재 57~61쪽 복습

값부터 눈으로 확인하는 습관

차가 이상하게 움직일 때 코드부터 고치지 말고, 센서 값 → 모터 입력 → 전원 순서로 확인합니다.

도구 → 시리얼 모니터 (Ctrl+Shift+M) / 시리얼 플로터 (Ctrl+Shift+L)

```
void setup() {
  Serial.begin(9600);          // 시리얼 모니터도 9600으로 맞출 것
}
void loop() {
  int l = analogRead(A5);      // 왼쪽 라인 센서
  int c = analogRead(A4);      // 가운데
  int r = analogRead(A3);      // 오른쪽
  Serial.print(l);  Serial.print(' ');
  Serial.print(c);  Serial.print(' ');
  Serial.println(r);           // 한 줄에 세 값 출력
  delay(100);
}
```

선 그래프로 그려집니다. 센서 값이 어떻게 변하는지 한눈에 볼 수 있습니다.

증상	먼저 볼 곳
바퀴가 안 돈다	전원 스위치, PWM 값이 너무 작은지, IN 핀 배선
한쪽만 돈다	그쪽 IN, EN 배선과 점퍼선 접촉
센서 값이 안 변한다	VCC, GND, 핀 번호, 센서와 바닥 간격
값이 튄다	배선 접촉, 배터리 전압
업로드가 안 된다	포트 선택, 0·1번 핀에 연결한 선이 있는지

공통 기초

delay 대신 millis

delay(500)을 부르면 0.5초 동안 아두이노는 아무것도 못 합니다. 그 사이 센서를 읽거나 블루투스 명령을 받을 수 없습니다.

- millis()는 아두이노가 켜진 뒤 지난 시간(ms)을 알려 주는 시계입니다 .
- '마지막으로 일한 시각'을 변수에 두고, 현재 시각과의 차이가 정해진 시간 이상이면 그때만 일합니다.
- 일하지 않는 동안에도 loop()는 계속 돌기 때문에 다른 일(블루투스 수신, 모터 제어)이 끊기지 않습니다.
- pulseIn은 최대 대기 시간(timeout)만큼 멈춥니다. 초음파에서는 25ms 정도로 제한합니다.
- 실습용 단독 테스트에서는 delay를 써도 됩니다. 통합 코드에서는 쓰지 않습니다.

코드 공통 2 일정 간격으로 일하기

```
unsigned long last = 0;

void loop() {
  unsigned long now = millis();
  if (now - last >= 100) {     // 100ms마다 한 번만
    last = now;
    // 여기서 센서를 읽는다
  }
  // 이 아래는 loop가 돌 때마다 계속 실행된다
  // (블루투스 수신, 모터 제어 등)
}
```

참고 now - last 뺄셈 방식은 millis 값이 한 바퀴 돌아 0으로 돌아가는 약 49일 뒤에도 문제없이 동작합니다.

교재 코드와 교안 코드 대응표

같은 일을 교재에서는 이렇게, 이 교안에서는 이렇게 씁니다. 두 코드를 비교하며 읽을 때 쓰세요.

하는 일	교재 예제	이 교안	달라지는 점
모터 핀	RightMotor_E/1/2_pin (5, 8, 9) LeftMotor_E/3/4_pin (6, 10, 11)	ENA, IN1, IN2 (5, 8, 9) ENB, IN3, IN4 (6, 10, 11)	번호는 같고 이름만 줄였습니다
전진	motor_role(HIGH, HIGH)	drive(150, 150)	교재는 방향(HIGH/LOW), 교안은 부호 있는 속도(-255~255)
제자리 회전	motor_role(LOW, HIGH) (우회전)	drive(120, -120)	오른쪽 바퀴 후진, 왼쪽 바퀴 전진
라인 센서	digitalRead(L_Line) == HIGH (A5, A4, A3)	analogRead(PIN_L) > TH	같은 센서입니다. 교안은 경계 TH를 직접 정합니다
기다리기	delay(500)	millis()로 시각 비교	기다리는 동안에도 센서와 통신을 계속 읽습니다
장애물	Ultra_d(mm) < 150 후진 150~250 서보로 좌우 확인	dist(cm) < 15 정지, > 20 출발	단위와 방식이 다릅니다. 두 기준으로 떨림을 막습니다
블루투스	SoftwareSerial bluetooth(3, 4) Blue_val g l r b s q w	SoftwareSerial BT(3, 4) cmd g l r b s q w + a, m	핀과 명령은 같고, 모드 전환(a, m)이 더해집니다

1

1팀

라인트레이싱 + 장애물 정지

교재 9번(라인트레이서)과 10번(장애물) 예제를 이미 실습했습니다. 이 프로젝트는 두 기능을 하나의 프로그램으로 합치고, 안정성을 숫자로 재서 개선합니다. 2팀과 4팀이 이 코드를 가져다 씁니다.

먼저 되어 있어야 하는 것

교재 ~200쪽(9~10번 예제)까지 실습한 상태

공통 · 모터 드라이버와 drive 함수

공통 · 핀 약속

차례

01 시작 전: 교재와 달라지는 것

02 라인 센서와 초음파 센서 원리

03 STEP 1 라인 센서 값 읽기

04 STEP 2 기준값(임계값) 정하기

05 STEP 3 초음파 거리 읽기

06 STEP 4 모터 단독 테스트

07 STEP 5 라인트레이싱

08 STEP 6 장애물 정지

09 STEP 7 안정성 측정

10 문제 해결

11 완료 기준과 발표 질문

12 심화 A 비례 제어

13 심화 B 영상 위에 값 표시

14 심화 C 서보로 좌우 확인

15 다음 단계

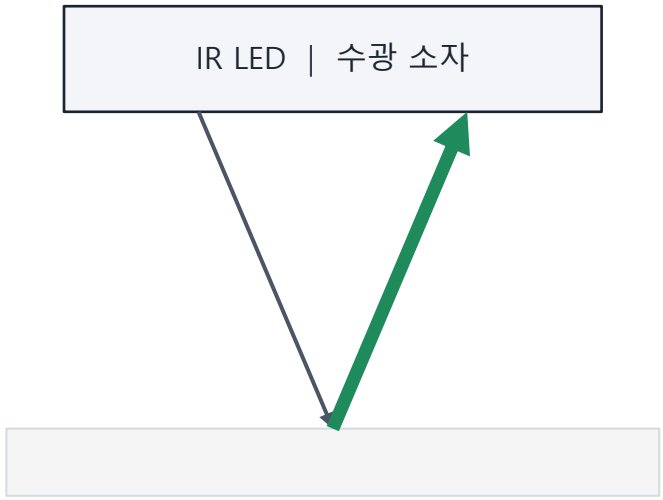
교재에서 한 것과 달라지는 것

교재 9번과 10번 예제는 이미 돌려 봤습니다. 1팀 프로젝트는 그 둘을 하나의 프로그램으로 합치고, 잘 되는지를 숫자로 재서 개선하는 것입니다.

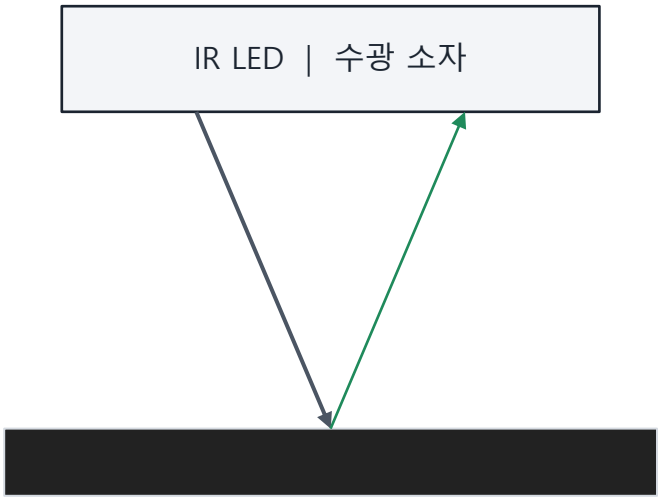
	교재에서 한 것	이 프로젝트에서 달라지는 것	이유
센서 읽기	digitalRead로 HIGH / LOW	analogRead 값을 기록하고 임계값 TH를 직접 정한다	조명이 바뀌어도 기준을 다시 잡을 수 있다
모터 제어	motor_role(방향) + 속도 변수	drive(left, right) 한 함수	곡선과 제자리 회전을 속도 두 개로 표현한다
기다림	delay()	millis()	기다리는 동안에도 센서를 계속 읽는다
장애물	150 / 250mm 기준, 후진 후 서보로 확인	15cm 정지, 20cm 재출발	정지와 출발이 반복되는 떨림을 막는다
확인 방법	눈으로 잘 되는지 본다	완주 시간, 이탈 횟수, 정지 거리를 기록한다	개선했다는 것을 숫자로 보인다

교재 쪽수 라인 센서 51~61쪽, 라인 주행 182~190쪽, 장애물 192~200쪽. 각 STEP 제목 옆의 '교재 ...' 표시가 같은 쪽수를 알려 줍니다.

라인 센서는 반사된 빛의 양을 잰다



흰 바닥
빛이 많이 돌아옴



검정 테이프
빛이 흡수되어 조금만 돌아옴

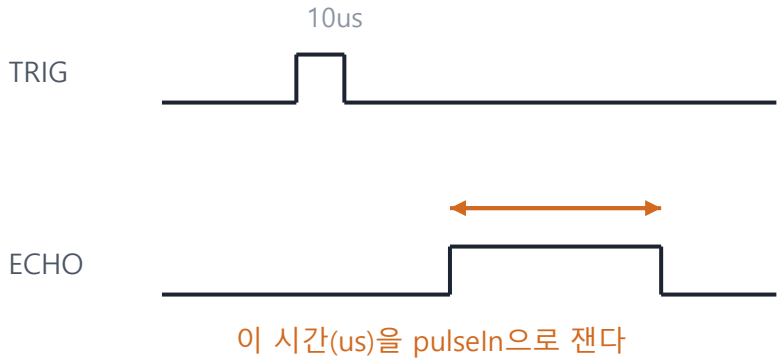
빛이 많이 돌아온 정도를 센서 보드가 전압으로 바꿔 줍니다.
어느 쪽에서 값이 커지는지는 모듈마다 다릅니다. 직접 읽어서 확인합니다.

그림. 라인 센서 단면 (개념도)

확인 검정에서 값이 커지는 모듈도, 작아지는 모듈도 있습니다. 코드의 부등호 방향을 정하기 전에 반드시 STEP 1에서 값을 읽어 확인합니다.

초음파 센서는 돌아온 시간으로 거리를 계산한다





거리(cm) = 시간(us) × 0.0343 ÷ 2

0.0343 cm/us 는 소리의 속도(약 343m/s). 갔다 오는 거리라서 2로 나눕니다.

라인 센서 값 읽기

목표

흰 바닥과 검정 선에서 센서 3개(왼쪽, 가운데, 오른쪽)의 값이 각각 얼마인지 숫자로 확인합니다.

할 일

- 코드를 올리고 시리얼 모니터(9600)를 엽니다.
- 센서 3개를 흰 바닥 위에 두고 값을 5번 적어 평균을 냅니다.
- 같은 방법으로 검정 테이프 위에서 적습니다.
- 센서마다 값이 조금씩 다른 것이 보통입니다. 한 개씩 손가락으로 가려 보면 어느 칸이 어느 센서인지 알 수 있습니다.

코드 1-1

```
const int PIN_L = A5;    // 왼쪽
const int PIN_C = A4;    // 가운데
const int PIN_R = A3;    // 오른쪽

void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.print("L=");   Serial.print(analogRead(PIN_L));
  Serial.print("  C="); Serial.print(analogRead(PIN_C));
  Serial.print("  R="); Serial.println(analogRead(PIN_R));
  delay(200);     // 값을 보기만 하는 단계라 delay를 써도 된다
}
```

	흰 바닥 (5회 평균)	검정 선 (5회 평균)
왼쪽 센서 (A5)		
가운데 센서 (A4)		
오른쪽 센서 (A3)		

확인

흰색과 검정의 값 차이가 눈에 띄게 큽니다. 교재 실험에서는 손가락으로 가린 센서가 10~25, 가리지 않은 센서가 700~900쯤 읽혔습니다. 차이가 작으면 센서와 바닥의 간격이나 조명을 먼저 확인합니다.

기준값(임계값) 정하기

먼저 되어 있어야 하는 것 STEP 1

- 임계값은 '이 값을 넘으면 검정 선 위에 있다'고 판단하는 경계입니다.
- 흰색 평균과 검정 평균의 **가운데**를 시작점으로 합니다.
- 센서 3개의 값 범위가 서로 다르면 TH_L, TH_C, TH_R을 따로 둡니다.
- 조명이 바뀌면(창가, 형광등, 시연장) 값이 바뀝니다. **시연하는 자리에서 다시 측정**합니다.
- 교재 예제(9_RC_CAR_Line)는 digitalRead로 HIGH/LOW만 읽습니다. 값이 중간(대략 300~600)에 걸리면 불안정할 수 있어서, 이 교안은 analogRead로 직접 정한 TH를 씁니다.

코드 1-2 판정 결과를 글자로 확인

```
const int TH = 450;                    // 직접 잰 값으로 바꾼다

bool onLine(int v) {
  return v > TH;                       // 검정에서 값이 클 때
}

void loop() {
  bool l = onLine(analogRead(A5));
  bool c = onLine(analogRead(A4));
  bool r = onLine(analogRead(A3));
  Serial.print(l ? "L:ON  " : "L:off ");
  Serial.print(c ? "C:ON  " : "C:off ");
  Serial.println(r ? "R:ON" : "R:off");
  delay(200);
}
```

예시 계산	
흰색 평균	180
검정 평균	720
임계값 (180 + 720) ÷ 2	450

확인

센서를 선 위에 올리면 ON, 흰 바닥에 올리면 off가 안정적으로 나옵니다. 경계에서 깜빡이면 임계값이나 센서 높이를 조정합니다.

초음파로 거리 읽기

코드 1-3

```
#include <Servo.h>
Servo myservo;
const int TRIG = 13, ECHO = 12;

long readCm() {
  digitalWrite(TRIG, LOW);   delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 25000);   // 25ms 안에 안 오면 포기
  if (us == 0) return 999;                // 측정 실패는 '멀다'로 처리
  return us * 0.0343 / 2;
}

void setup() {
  pinMode(TRIG, OUTPUT);  pinMode(ECHO, INPUT);
  myservo.attach(2);  myservo.write(90);   // 센서를 정면(90도)으로
  Serial.begin(9600);
}
void loop() {
  Serial.println(readCm());
  delay(100);
}
```

초음파 센서가 잘 못 재는 경우

- 이 키트의 초음파 센서는 **서보모터(2번 핀) 위**에 달려 있습니다. 앞을 보려면 서보를 90도로 둡니다.
- 측정 범위는 대략 2cm~4m입니다. 너무 가까워도 안 됩니다.
- 센서를 정면으로 마주 보지 않는 비스듬한 면은 소리가 엉뚱한 쪽으로 튕겨 갑니다.
- 옷, 스펀지처럼 부드러운 물체는 소리를 흡수해서 잘 안 잡힙니다.
- 측정 각도가 넓지 않습니다(대략 15도). 장애물은 정면에 두고 시험합니다.
- 읽은 값이 999라면 측정 실패입니다.

실제 거리(줄자)	10cm	20cm	30cm	50cm
측정값 1회				
측정값 2회				

확인: 줄자와 비교해서 오차가 1~2cm 안쪽입니다.

모터 단독 테스트

먼저 되어 있어야 하는 것 **공통 · 모터 드라이버와 drive 함수**

먼저 차체 아래에 상자를 받쳐 **바퀴가 바닥에 닿지 않게** 합니다. 책상에서 차가 달려 떨어지지 않도록 합니다.

코드 1-4 (setMotor, drive는 공통 기초의 코드를 그대로 넣습니다)

```
void loop() {
  Serial.println("forward");   drive(150, 150);    delay(1500);
  Serial.println("stop");      drive(0, 0);        delay(800);
  Serial.println("backward");  drive(-150, -150);  delay(1500);
  Serial.println("stop");      drive(0, 0);        delay(800);
  Serial.println("turn left"); drive(-120, 120);   delay(1200);
  Serial.println("turn right");drive(120, -120);   delay(1200);
  Serial.println("stop");      drive(0, 0);        delay(1500);
}
```

최소 구동값 찾기

drive(60, 60), drive(80, 80), drive(100, 100)처럼 값을 조금씩 올리면서 **바퀴가 겨우 돌기 시작하는 값**을 찾아 적습니다. 이 값 아래로는 속도를 주지 않습니다. (교재 예제는 153으로 시작합니다.)

	왼쪽	오른쪽
돌기 시작하는 값		

방향이 이상할 때

- 한쪽만 반대로 돈다 → 그쪽 IN 두 선을 서로 바꾸거나 코드에서 그쪽 부호를 뒤집는다.
- 양쪽 모두 반대로 돈다 → 양쪽 다 같은 방법으로 바꾼다.
- '왼쪽 회전'인데 오른쪽으로 돈다 → 좌우가 바뀐 것이므로 ENA/IN1·2와 ENB/IN3·4를 서로 바꾼다.

확인: 전진·후진·좌우 회전이 이름대로 움직입니다.

라인트레이싱

선이 가운데 센서 아래에 오도록 차를 올려놓습니다. 양옆 센서 중 하나가 선에 닿으면 그쪽으로 차를 돌려서 선을 다시 가운데로 가져옵니다.

먼저 되어 있어야 하는 것 STEP 2 STEP 4

왼쪽	가운데	오른쪽	의미	동작
흰	검정	흰	선이 가운데	직진
검정	무관	흰	선이 왼쪽으로 치우침	왼쪽으로 돈다
흰	무관	검정	선이 오른쪽으로 치우침	오른쪽으로 돈다
검정	무관	검정	교차선 또는 선의 끝	정지
흰	흰	흰	선을 놓침	직전 동작 유지

- BASE(기본 속도)는 낮게(100 안팎) 시작해서 올립니다. 빠르면 선을 놓칩니다.
- 커브에서 선을 놓치면 SLOW를 더 작게, 0이나 음수로 줍니다.
- 센서 3개 사이가 선 폭보다 많이 넓으면 선이 센서 사이로 빠집니다. 센서 간격을 확인합니다.

확인: 직선과 완만한 곡선 트랙을 한 바퀴 돕니다.

코드 1-5 이 lineTrace() 함수를 2팀, 4팀이 그대로 가져다 씁니다

```
const int TH = 450;      // 직접 잰 값
const int BASE = 120;    // 기본 속도
const int SLOW = 30;     // 안쪽 바퀴 속도
bool lastL = false, lastC = true, lastR = false;

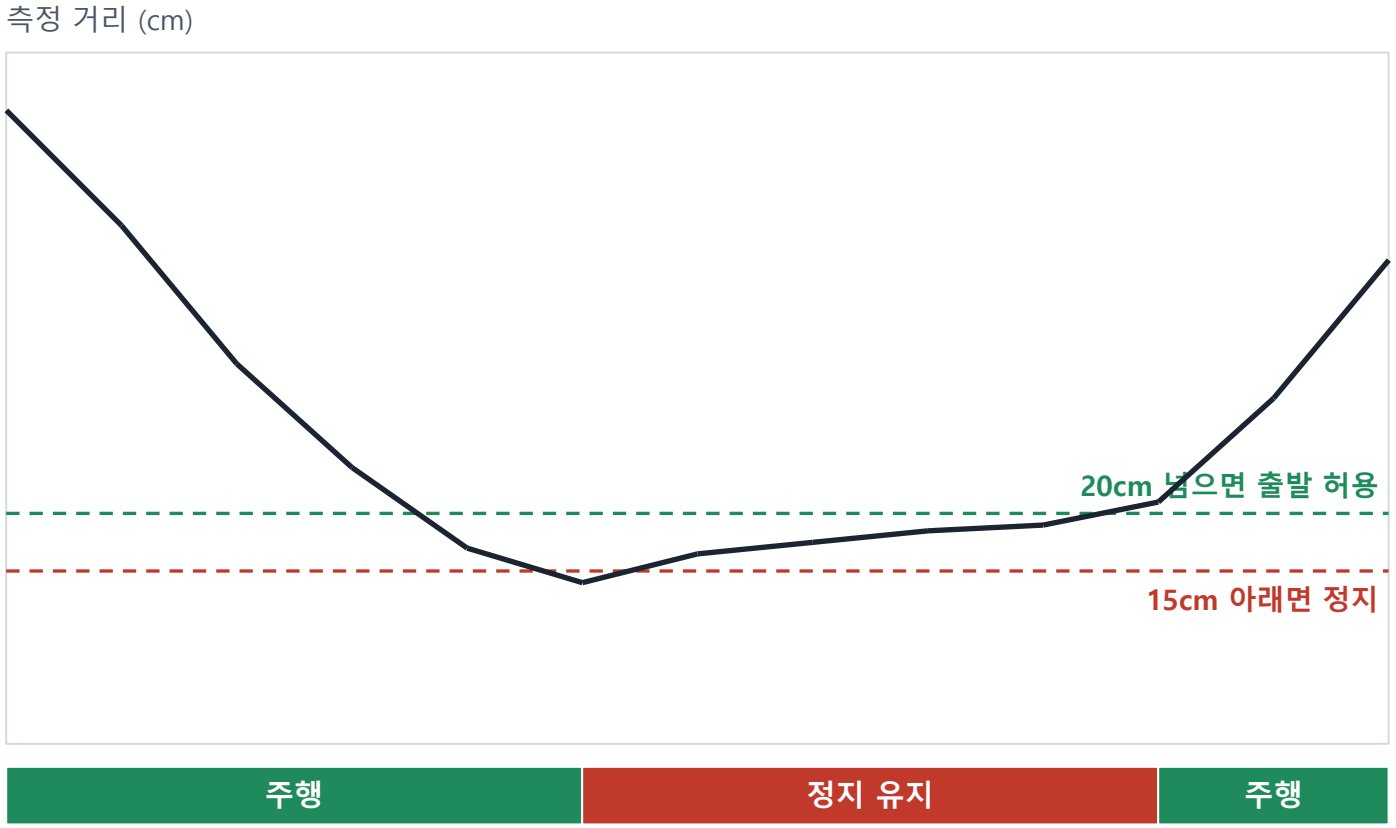
void lineTrace() {
  bool l = analogRead(PIN_L) > TH;   // 검정 선 위
  bool c = analogRead(PIN_C) > TH;
  bool r = analogRead(PIN_R) > TH;

  if (!l && !c && !r) {              // 선을 놓침: 직전 값을 쓴다
    l = lastL;  c = lastC;  r = lastR;
  } else {
    lastL = l;  lastC = c;  lastR = r;
  }

  if (l && r)    drive(0, 0);          // 교차선 / 끝
  else if (l)    drive(SLOW, BASE);    // 왼쪽으로 돈다
  else if (r)    drive(BASE, SLOW);    // 오른쪽으로 돈다
  else           drive(BASE, BASE);    // 가운데만 -> 직진
}

void loop() {
  lineTrace();
}
```

장애물 정지: 기준을 두 개 두는 이유



그림의 15cm 선 하나만 있었다면 6번 시점에서 다시 출발했을 것입니다.

- 15cm 하나만 쓰면 거리가 14.8과 15.2 사이에서 튈 때 **정지와 출발이 계속 반복**되어 차가 떱니다.
- 그래서 정지는 15cm 아래, 출발은 **20cm를 넘어야** 하도록 기준을 둘로 나눕니다. 이런 방식을 히스테리시스라고 부릅니다.
- 정지한 뒤 최소 1초는 멈춰 있게 합니다. 순간적인 측정 오류에 반응하지 않기 위해서입니다.
- 장애물 판단이 라인트레이싱보다 **우선**입니다. if 문의 순서로 우선순위를 표현합니다.

장애물 정지 코드 (최종 loop)

먼저 되어 있어야 하는 것 **STEP 3 readCm** **STEP 5 lineTrace**

코드 1-6

```
enum State { DRIVING, STOPPED };
State state = DRIVING;

const int STOP_CM = 15, RESUME_CM = 20;
const unsigned long HOLD_MS = 1000;      // 정지 후 최소 유지 시간
unsigned long lastPing = 0, stoppedAt = 0;
long dist = 999;

void loop() {
  unsigned long now = millis();

  if (now - lastPing >= 60) {            // 60ms마다 거리 측정
    lastPing = now;
    dist = readCm();                     // STEP 3의 함수
  }

  if (state == DRIVING && dist < STOP_CM) {
    state = STOPPED;  stoppedAt = now;
  } else if (state == STOPPED && dist > RESUME_CM && now - stoppedAt > HOLD_MS) {
    state = DRIVING;
  }

  if (state == STOPPED) drive(0, 0);     // 장애물이 가장 우선
  else                  lineTrace();     // STEP 5의 함수
}
```

읽는 순서

- 거리는 60ms마다만 잰다(millis).
- 상태(state)는 DRIVING과 STOPPED 두 가지뿐이다.
- 정지 조건과 출발 조건이 다르다(15cm / 20cm).
- 마지막 if에서 정지 상태를 먼저 검사한다.

확인

선을 따라 달리다 상자 앞 15cm에서 멈추고, 상자를 치우면 1초 이상 지난 뒤 다시 출발합니다.

안정성 측정: 개선을 숫자로 보이기

교재 예제는 '잘 된다'를 눈으로 확인했습니다. 이 프로젝트에서는 같은 트랙에서 세 번씩 재서 기록합니다.

먼저 되어 있어야 하는 것 **STEP 6**

항목	재는 방법	1회	2회	3회
완주 시간 (초)	출발선에서 끝선까지			
선 이탈 횟수	바퀴가 선 밖으로 나간 횟수			
장애물 정지 거리 (cm)	멈춘 뒤 장애물과 거리(줄자)			
재출발 시간 (초)	장애물을 치운 뒤 출발까지			
떨림 횟수	정지와 출발이 반복된 횟수			

분석 질문: 가장 큰 개선은 어느 값을 바꿨을 때였나요? 그 이유는 무엇이라고 생각하나요?

측정하는 요령

- 조명, 트랙, 배터리 상태를 같게 하고 3번 반복합니다.
- 개선하기 전(STEP 5 코드)과 후(STEP 6 코드)를 따로 기록해서 비교합니다.
- 임계값, BASE, SLOW를 바꿀 때마다 노트에 값과 결과를 적습니다.

1팀

자주 막히는 곳

증상	원인	조치
선 위에서 계속 한쪽으로 쏠린다	모터 배선 또는 IN 핀이 바뀜	STEP 4에서 좌우 방향 다시 확인
커브에서 선을 놓친다	BASE가 크거나 SLOW가 크다	BASE 낮추기, SLOW를 0이나 음수로
직선에서 좌우로 흔들린다	임계값이 경계에 걸림, 센서 높이	임계값 다시 측정, 센서 높이 맞추기
장애물 앞에서 덜덜 떤다	정지·출발 기준이 하나뿐	STOP_CM과 RESUME_CM 분리, HOLD_MS 추가
거리 값이 999로 자주 나온다	반사면이 비스듬하거나 흡수	평평한 상자로 시험, 센서 방향 확인
켜자마자 멈춘다	센서 앞이 가려졌거나 초기값 문제	센서 앞 확인, dist 초기값 999 확인
배터리가 약해지면 안 간다	L298N 전압 강하	충전된 배터리로 교체, 최소 구동값 다시 확인

1팀

완료 기준과 발표 질문

완료 기준

- ☐ 센서 3개의 흰색/검정 값을 기록했다
- ☐ 임계값을 정했고 이유를 말할 수 있다
- ☐ 모터가 이름대로 움직이고 최소 구동값을 안다
- ☐ 직선과 곡선 트랙을 한 바퀴 돈다
- ☐ 장애물 앞에서 멈추고 치우면 다시 출발한다
- ☐ delay 없이 millis로 거리 측정을 돌린다

발표 때 받게 될 질문

- 임계값은 왜 이 숫자로 정했나요? 측정 기록을 보여 주세요.
- 선이 가운데 센서 아래에 있을 때 왜 직진인가요?
- 정지 기준(15cm)과 출발 기준(20cm)이 왜 다른가요?
- delay 대신 millis를 쓴 이유가 무엇인가요?
- 속도를 올리면 어떤 문제가 생기나요? 어떻게 해결했나요?

1팀 심화 A

비례 제어(P제어)로 부드럽게

켜짐/꺼짐으로만 판단하던 것을, 센서 값의 크기에 비례해서 조금씩 꺾도록 바꿉니다.

먼저 되어 있어야 하는 것 **STEP 5**

- 오차(error) = 왼쪽 값 − 오른쪽 값. 가운데 센서는 쓰지 않습니다. 선이 왼쪽 센서에 가까울수록 커집니다.
- 오차에 Kp를 곱한 만큼 좌우 속도에 차이를 줍니다.
- 선이 한쪽 센서에 닿기 시작하는 순간부터 닿은 정도에 비례해서 조향합니다. 선이 가운데에 있으면 양옆 값이 비슷해서 오차가 0에 가까워 직진합니다.

코드 1-7 (검정에서 값이 커지는 센서 기준)

```
float Kp = 0.15;

void lineTracePID() {
  int l = analogRead(PIN_L);
  int r = analogRead(PIN_R);
  int error = l - r;                  // 왼쪽이 검정에 가까우면 +
  int corr  = Kp * error;
  drive(BASE - corr, BASE + corr);  // 왼쪽 늦춤, 오른쪽 빠름
}
```

튜닝 순서

- Kp를 0에서 시작 (직진만)
- 0.05씩 올려 커브를 통과하는지 본다
- 차가 좌우로 계속 흔들리면 한 단계 줄인다
- BASE를 올리고 다시 반복

비교 STEP 5(켜짐/꺼짐)와 같은 트랙에서 완주 시간과 흔들림을 비교해서 노트에 적어 보세요.

1팀 심화 B

영상 위에 차량 값 표시하기

3팀이 만든 영상 보기 코드에 아두이노가 보내는 거리값과 상태를 글자로 얹습니다. 색 인식은 하지 않고, 내가 제어하는 값을 눈으로 확인하는 것이 목적입니다.

먼저 되어 있어야 하는 것 **3팀 STEP 3 영상 보기**

아두이노 쪽

```
// 아두이노: 100ms마다 한 줄 전송 (STEP 6 loop에 추가)
unsigned long lastTx = 0;
...
if (now - lastTx >= 100) {
  lastTx = now;
  Serial.print(dist);  Serial.print(',');
  Serial.println(state == STOPPED ? "STOP" : "GO");
}
```

PC 쪽

```
# PC (파이썬): 영상을 읽으면서 시리얼도 읽는다
import cv2, serial
ser = serial.Serial("COM3", 9600, timeout=0.01)   # 아두이노 USB
cap = cv2.VideoCapture("http://192.168.0.10:81/stream")  # IP 변경
text = "waiting"
while True:
    ok, frame = cap.read()
    if not ok: break
    line = ser.readline().decode(errors="ignore").strip()   # 예: 23,GO
    parts = line.split(",")
    if len(parts) == 2:
        text = f"dist {parts[0]}cm  {parts[1]}"
    cv2.putText(frame, text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
                0.9, (0, 255, 0), 2)
    cv2.imshow("telemetry", frame)
    if cv2.waitKey(1) == 27: break
```

주의 cv2.putText는 한글을 그리지 못합니다(물음표로 나옴). 영어와 숫자로 표시합니다. 아두이노가 USB로 PC에 연결되어 있어야 하므로 시험은 차량을 케이블 길이 안에서 움직입니다.

1팀 심화 C 교재 193~197쪽 참고

서보로 좌우를 확인하기

초음파 센서는 서보모터 위에 달려 있습니다. 장애물 때문에 멈춘 뒤 센서를 좌우로 돌려 어느 쪽이 더 트였는지 재 봅니다. 가벼운 단계부터 시작합니다.

먼저 되어 있어야 하는 것 **STEP 6**

1단계 재기만 하기 (권장)

- 장애물 앞에서 정지한 상태에서만 실행합니다. 차가 서 있으므로 delay를 써도 됩니다.
- 30도와 150도에서 거리를 재고, 90도로 되돌립니다. 각도는 교재 예제와 같습니다.
- 결과를 시리얼 모니터에 출력만 합니다. 차는 움직이지 않습니다.

2단계 비켜가기 (도전, 평가 제외)

- 더 트인 쪽으로 돌고, 조금 가서 반대로 돌아 선을 다시 찾아 돌아옵니다.
- 선을 놓치는 문제가 새로 생기므로 난이도가 확 올라갑니다.

코드 1-8 1단계

```
#include <Servo.h>
Servo myservo;                    // setup()에서 myservo.attach(2);

long scanAt(int angle) {          // 그 각도의 거리(cm)
  myservo.write(angle);
  delay(500);                     // 서보가 돌 시간 (차는 정지 상태)
  return readCm();
}

// 장애물 때문에 STOPPED가 된 순간에 한 번 실행
void scanSides() {
  long a = scanAt(30);
  long b = scanAt(150);
  myservo.write(90);              // 정면으로 복귀
  Serial.print("30도: ");   Serial.print(a);
  Serial.print("   150도: "); Serial.println(b);
}
```

확인 30도 쪽이 차의 왼쪽인지 오른쪽인지는 조립에 따라 다를 수 있습니다. 교재 1번 예제(0, 90, 180도)로 먼저 눈으로 확인합니다.

다음 단계: 이 코드를 다른 팀이 이렇게 씁니다

이어 갈 팀	1팀 코드가 쓰이는 곳	바로 가기
2팀	자동 모드에서 lineTrace()를 그대로 호출	2팀 STEP 5 모드 전환
4팀	'g' 신호일 때 lineTrace(), 's'일 때 정지. 장애물 정지도 그대로 유지	4팀 STEP 3 차량 수신
3팀	(심화 B) 영상 위에 거리·상태 표시	3팀 STEP 3 영상 보기

2팀 STEP 5 모드 전환 **4팀 STEP 3 차량 수신** **3팀 STEP 3 영상 보기**

1팀이 먼저 끝났다면 STEP 5와 6의 코드를 그대로 두고 2팀의 블루투스 모듈을 추가해 보세요. 자동 모드는 이미 완성되어 있고, 명령을 받아서 모드를 바꾸는 부분만 새로 만들면 됩니다.

2

2팀

블루투스 수동/자동 모드 전환

스마트폰 앱으로 차를 조종하다가 버튼 하나로 자동(라인트레이싱) 모드로 넘기고, 다시 수동으로 되돌리는 시스템을 만듭니다. 블루투스는 교재 부록을 먼저 따라 합니다. 4팀이 이 수신 코드를 가져다 씁니다.

먼저 되어 있어야 하는 것

교재 부록 203쪽~ 블루투스 (아직 안 한 부분)

공통 · 모터 드라이버와 drive 함수

1팀 STEP 5 라인트레이싱

차례

01 시작 전: 교재 부록 먼저

02 블루투스 원리와 모듈 종류

03 배선

04 STEP 1 에코 테스트

05 STEP 2 명령 약속 정하기

06 STEP 3 수동 주행

07 STEP 4 앱 만들기

08 STEP 5 수동/자동 모드 전환

09 STEP 6 안전장치

10 문제 해결과 완료 기준

11 심화 양방향, 1인칭 시점

12 다음 단계

2팀 시작 전

교재 부록을 먼저 따라 하세요

블루투스는 교재 203쪽부터의 부록에 있고 아직 하지 않은 부분입니다. 아래 순서로 교재를 먼저 따라 하면 STEP 1~4와 같은 내용이라 프로젝트가 수월해집니다.

교재 쪽수	교재 내용	이 교안에서 이어지는 곳
208~211쪽	블루투스 결선 (센서 확장 쉴드 S(3), S(4))	배선
212~221쪽	11_Bluetooth 예제, AT 명령으로 통신 확인	STEP 1 에코 테스트
222~234쪽	HC-06 페어링, 앱 설치, AIA 파일 불러오기, 연결	STEP 4 앱 만들기
235~242쪽	12_RC_CAR_Bluetooth 코드, g l r b s q w 명령	STEP 2~3 명령 약속, 수동 주행
243~249쪽	HM-10(BLE) 모듈 (아이폰용)	원리 슬라이드의 아이폰 안내

새로 하는 것 교재까지는 앱으로 조종만 합니다. 이 프로젝트에서는 수동/자동 모드 전환(a, m), 연결이 끊기면 멈추는 안전장치, 앱에 전환 버튼 추가를 새로 합니다.

2팀 원리

블루투스 모듈은 '선 없는 시리얼'이다

- USB 케이블로 시리얼 모니터에 글자를 보내던 것과 같습니다. 폰이 'g'를 보내면 아두이노는 bluetooth.read()로 'g'를 받습니다.
- **HC-05, HC-06**은 칩 이름이고 **ZS-040**은 그 칩을 올려 놓은 보드 이름입니다. 사진이 비슷한 이유입니다.
- 이 프로젝트는 폰이 먼저 연결을 거는 구조(폰=마스터, 차=슬레이브)라서 교재처럼 HC-06을 그대로 쓰면 편합니다.
- 연결 비밀번호는 보통 0000 또는 1234, 통신 속도는 교재 예제와 같이 9600입니다. 모듈은 키트에 들어 있지 않은 **별매품**입니다.

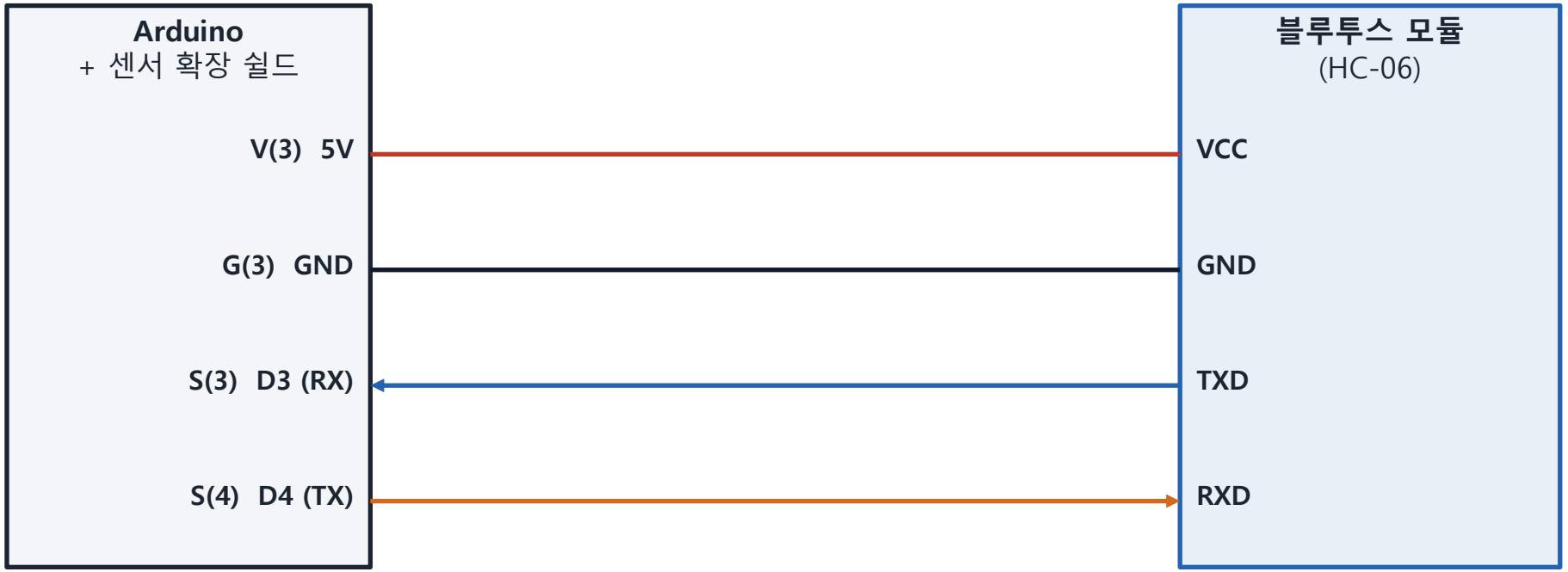
	HC-05	HC-06
역할	마스터·슬레이브 전환 가능	슬레이브 전용
설정	AT 명령으로 모드 지정	페어링만 하면 바로 사용
구분법	칩에 인쇄된 글자, 폰에 뜨는 기기 이름으로 확인	같음

참고 HC-05를 샀다면 처음 한 번만 AT 명령으로 슬레이브 모드로 맞추면 이후엔 HC-06과 똑같이 씁니다.

아이폰이면 HC-06은 클래식 블루투스(SPP)라서 **아이폰에서는 쓸 수 없습니다.** 교재도 안드로이드는 HC-06, 아이폰은 HM-10(BLE)을 쓰라고 안내합니다. 다만 교재의 앱인벤터 실습은 안드로이드와 HC-06 기준이고, HM-10은 앱인벤터 실습에서 안드로이드로 쓸 수 없다고 되어 있습니다.

배선: TX와 RX는 엇갈리게

아두이노의 0, 1번 핀(USB와 공용)을 쓰면 코드 업로드 때 충돌하므로, 교재처럼 3, 4번 핀에 SoftwareSerial을 만들어 씁니다.



모듈이 보내는 쪽(TXD)을 아두이노가 받는 쪽(D3)에, 아두이노가 보내는 쪽(D4)을 모듈이 받는 쪽(RXD)에 연결합니다. 서로 엇갈리게 연결하는 것이 정상입니다.

참고 교재는 센서 확장 쉴드의 S(3), S(4) 소켓에 점퍼선으로 직결합니다. RXD는 3.3V 기준이라 1kΩ과 2kΩ 저항으로 분압하면 모듈에 더 안전하지만 필수는 아닙니다.

에코 테스트: 폰과 아두이노가 글자를 주고받는지

- 코드를 올린 뒤 폰의 블루투스 설정에서 모듈(HC-06)을 찾아 **페어링**합니다(1234).
- 안드로이드 앱 **Serial Bluetooth Terminal**처럼 시리얼 터미널 앱으로 모듈에 연결합니다.
- 폰에서 g를 보내면 아두이노 시리얼 모니터(9600)에 g가 보여야 합니다.
- 시리얼 모니터에서 글자를 보내면 폰 앱에도 보이는지 확인합니다.

코드 2-1

```
#include <SoftwareSerial.h>
#define BT_RXD 3
#define BT_TXD 4
SoftwareSerial BT(BT_RXD, BT_TXD);

void setup() {
  Serial.begin(9600);
  BT.begin(9600);
}

void loop() {
  // 폰 -> 모니터
  if (BT.available())  Serial.write(BT.read());
  // 모니터 -> 폰
  if (Serial.available())  BT.write(Serial.read());
}
```

확인

양쪽으로 글자가 오갑니다. 모듈의 LED 깜빡임이 연결 전후에 달라지는 것도 단서입니다(제품마다 패턴이 다름).

안 되면 연결은 되는데 글자가 안 오가면 TX/RX를 서로 바꿔 꽂아 보고, 글자가 깨지면 통신 속도(9600)를 의심합니다.

명령 약속 정하기

문자	의미	차량 동작
`g`	전진	drive(150, 150)
`b`	후진	drive(-150, -150)
`l`	좌회전 (곡선)	drive(60, 150)
`r`	우회전 (곡선)	drive(150, 60)
`s`	정지	drive(0, 0)
`q`	제자리 좌회전	drive(-120, 120)
`w`	제자리 우회전	drive(120, -120)
`a`	자동 모드로 전환 (새로 추가)	라인트레이싱 시작
`m`	수동 모드로 전환 (새로 추가)	정지 후 조종 대기

약속을 만드는 요령

- g, b, l, r, s, q, w는 교재의 조종 앱(Eduino_RC_Car_Controller)이 보내는 글자와 같습니다. a와 m이 이 프로젝트에서 새로 만드는 명령입니다.
- 한 글자 명령은 받는 코드가 가장 단순합니다.
- 버튼을 **누르고 있는 동안** 같은 명령을 반복해서 보내고, 떼면 s를 보냅니다.
- 앱과 아두이노가 **같은 표**를 보고 만들어야 합니다. 이 표를 팀 노트 첫 장에 적어 둡니다.

수동 주행: 받은 글자대로 움직이기

먼저 되어 있어야 하는 것 **공통 · drive 함수** **STEP 1**

코드 2-2

```
#include <SoftwareSerial.h>
#define BT_RXD 3
#define BT_TXD 4
SoftwareSerial BT(BT_RXD, BT_TXD);
char cmd = 's';

void driveManual(char c) {
  switch (c) {
    case 'g': drive( 150,  150); break;   // 전진
    case 'b': drive(-150, -150); break;   // 후진
    case 'l': drive(  60,  150); break;   // 좌회전(곡선)
    case 'r': drive( 150,   60); break;   // 우회전(곡선)
    case 'q': drive(-120,  120); break;   // 제자리 좌회전
    case 'w': drive( 120, -120); break;   // 제자리 우회전
    default:  drive(0, 0);                // 's' 포함
  }
}

void setup() {
  BT.begin(9600);
  // 모터 핀 설정은 공통 기초의 setup 참고
}

void loop() {
  if (BT.available()) cmd = BT.read();
  driveManual(cmd);
}
```

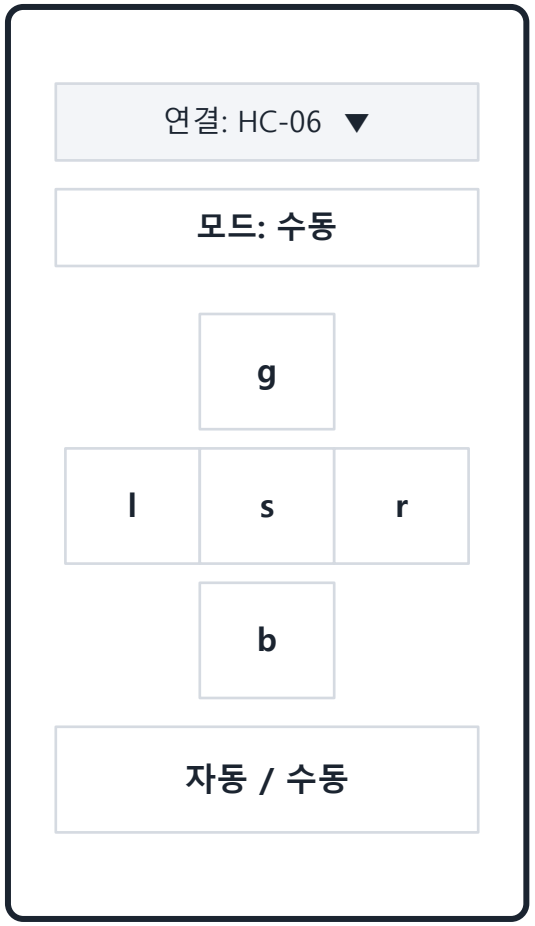
- 앱을 만들기 **전에** 터미널 앱으로 g, s, l, r을 보내 보면서 차가 반응하는지 먼저 봅니다.
- 차체를 들고(바퀴가 떠 있는 상태에서) 시험합니다.
- 마지막으로 받은 명령(cmd)이 계속 남아 있다는 점을 기억해 둡니다. STEP 6에서 이 문제를 다룹니다.

확인

터미널 앱에서 보낸 글자대로 차가 앞뒤, 좌우로 움직이고 s에서 멈춥니다.

앱 만들기 (1) 교재 앱을 불러와서 고치기

교재가 제공하는 앱 파일(Eduino_RC_Car_Controller, AIA)을 MIT App Inventor(ai2.appinventor.mit.edu)에 불러옵니다. 폰에는 'MIT AI2 Companion' 앱을 설치합니다.



구분	이름(예시)	하는 일
교재 앱에 있음	Bluetooth Connect 버튼	블루투스 기기를 골라 연결
교재 앱에 있음	조종 버튼 7개	g, l, s, r, b, q, w 전송
새로 추가	Button_mode	'a' 또는 'm' 전송 (자동 / 수동)
새로 추가	Label_mode	현재 모드 표시
새로 추가	Clock1 (TimerInterval 200)	누르는 동안 명령 반복 전송

참고 컴포넌트 이름은 AIA 파일을 열어서 실제 이름을 확인하세요. 앱 파일은 키트 제공 자료의 '[6] 제공 어플 - AIA파일' 폴더에 있습니다. 폰은 Wi-Fi에 연결되어 있어야 합니다.

안드로이드 12 이상 앱이 '근처 기기' 권한(BLUETOOTH_CONNECT, BLUETOOTH_SCAN)을 받아야 연결할 수 있습니다. 권한 창이 뜨면 허용합니다.

앱 만들기 (2) 블록

먼저 되어 있어야 하는 것 **STEP 2 명령 약속**

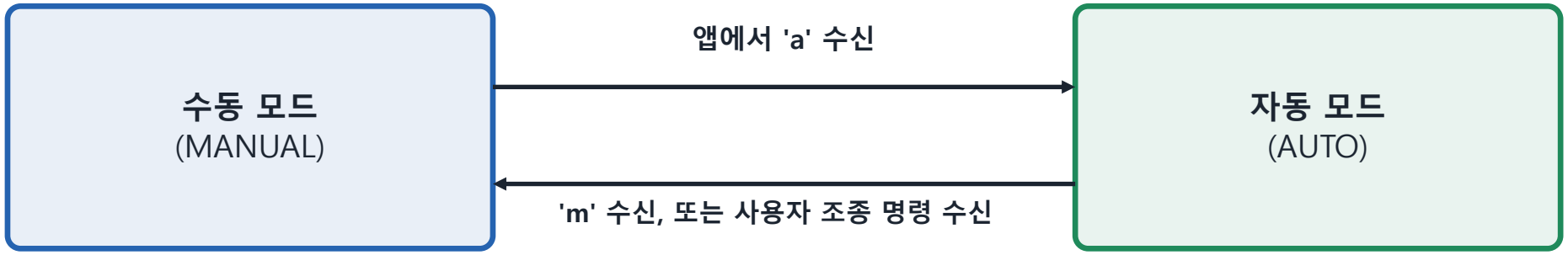
이벤트	하는 일
Bluetooth Connect 버튼	교재 앱에 이미 구현되어 있습니다. 기기 목록(AddressesAndNames)에서 HC-06을 골라 연결합니다.
전진 버튼.TouchDown	전역변수 cmd ← "g" 그리고 BluetoothClient1.SendText("g")
전진 버튼.TouchUp	전역변수 cmd ← "s" 그리고 BluetoothClient1.SendText("s")
(b, l, r, q, w 버튼도 같은 방식)	TouchDown에 해당 글자, TouchUp에 s
Clock1.Timer (200ms)	cmd가 "s"가 아니면 SendText(cmd) (누르고 있는 동안 반복 전송)
Button_mode.Click	지금 "수동"이면 SendText("a"), 아니면 SendText("m") 그리고 Label_mode 글자 바꾸기

참고 교재 앱의 버튼이 TouchDown/TouchUp 방식인지 Click 방식인지 먼저 확인합니다. Click은 손가락을 **뗄 때** 한 번 발생합니다. 누르는 동안 움직이게 하려면 TouchDown과 TouchUp을 씁니다. 모드 표시 글자는 앱이 추정한 값이라 차의 실제 모드와 어긋날 수 있습니다(심화에서 보완).

수동/자동 모드: 상태 하나로 기억하기

모드는 변수 하나입니다. 모드에 따라 loop()에서 부르는 함수만 달라집니다.

먼저 되어 있어야 하는 것 **1팀 STEP 5 lineTrace()** **STEP 3**



g b l r s q w 명령을 받아서
그대로 모터를 움직인다

1팀의 라인트레이싱 함수를
그대로 호출한다

전환할 때마다 모터를 멈추고 0.5초 동안 기다린다

모드 전환 코드

먼저 되어 있어야 하는 것 **STEP 5 그림**

코드 2-3

```
enum Mode { MANUAL, AUTO };
Mode mode = MANUAL;
char cmd = 's';
unsigned long holdUntil = 0;

void setMode(Mode m) {
  mode = m;  cmd = 's';
  drive(0, 0);
  holdUntil = millis() + 500;          // 전환 직후 0.5초는 무조건 정지
  BT.write(m == AUTO ? 'a' : 'm');     // 현재 모드를 폰에 알려 줌
}

void loop() {
  while (BT.available()) {
    char c = BT.read();
    if      (c == 'a') setMode(AUTO);
    else if (c == 'm') setMode(MANUAL);
    else {
      if (mode == AUTO) setMode(MANUAL);   // 자동 중 조종 명령 = 사람이 개입
      cmd = c;
    }
  }
  if (millis() < holdUntil) { drive(0, 0); return; }
  if (mode == AUTO) lineTrace();           // 1팀 함수 그대로
  else              driveManual(cmd);
}
```

눈여겨볼 곳

- 자동 모드에서 조종 명령이 오면 **사람이 우선**이라 수동으로 돌아갑니다.
- 전환할 때마다 정지와 0.5초 대기를 넣어 갑작스러운 움직임을 막습니다.
- delay 없이 holdUntil(시각)로 기다립니다.
- 버튼에서 손가락을 뗄 때 오는 s가 자동 전환 직후에 섞이면 수동으로 되돌아갈 수 있습니다. 시험하면서 확인하세요.

안전장치: 연결이 끊기면 멈춘다

폰이 꺼지거나 범위를 벗어나도 마지막 명령(g)이 남아 있으면 차는 계속 달립니다. 앱은 누르는 동안 명령을 반복 전송하고, 차는 일정 시간 명령이 없으면 멈추도록 합니다.

먼저 되어 있어야 하는 것 **STEP 5**

코드 2-4 수동 모드의 끊김 대비

```
unsigned long lastRx = 0;
const unsigned long LINK_MS = 600;   // 이 시간 무명령이면 정지

// loop() 안, BT.read() 직후에 추가
lastRx = millis();

// 마지막 줄 driveManual(cmd) 앞에 추가
if (millis() - lastRx > LINK_MS) cmd = 's';
driveManual(cmd);
```

코드 2-5 선택: 안전 기능이 사용자 명령보다 우선

```
// 수동 모드에서도 앞에 장애물이 있으면 전진하지 않는다
// (1팀의 dist, STOP_CM 사용)
if (mode == MANUAL && cmd == 'g' && dist < STOP_CM) cmd = 's';
```

- 앱의 Clock1이 200ms마다 같은 명령을 다시 보내야 합니다(STEP 4 블록).
- LINK_MS(600ms)는 앱의 반복 주기(200ms)보다 충분히 커야 합니다. 너무 작으면 정상 연결에서도 끊김으로 오해합니다.
- 일부러 폰의 블루투스를 꺼서 차가 멈추는지 시험합니다.

안전 기능을 만들었다면 반드시 일부러 고장을 내서 시험해 봅니다.

2팀

자주 막히는 곳과 완료 기준

증상	원인	조치
앱의 기기 목록이 비어 있다	폰에서 페어링 전, 또는 권한 거부	폰 설정에서 먼저 HC-06을 페어링(0000 또는 1234), 안드로이드 12 이상은 근처 기기 권한 허용
연결은 되는데 차가 반응 없다	TX/RX를 안 엇갈림, 속도 불일치, 전원	TX와 RX를 엇갈려 연결, 9600 확인, 모듈 전원(VCC)
글자가 깨진다	통신 속도가 다름	모듈 기본 속도 확인(9600 외 값도 있음)
가끔 명령이 사라진다	모터 노이즈로 전원 불안정	모터 전원선과 모듈 선을 떼어 배선, 반복 전송으로 보완
코드 업로드가 안 된다	0, 1번 핀에 연결해 둠	우리는 3, 4번 사용. 모듈 선을 0, 1번에서 뺀다
앱의 모드 표시가 차와 다르다	앱이 추정한 값	차가 보내는 a/m으로 라벨을 갱신(심화)

완료 기준

- ☐ 터미널 앱으로 글자가 양방향 오간다
- ☐ 앱 버튼으로 전후좌우가 움직인다
- ☐ a를 보내면 라인을 따라간다
- ☐ 자동 중 조종하면 수동으로 돌아온다
- ☐ 폰을 꺼도 차가 멈춘다

양방향 통신, 1인칭 시점 조종

A 차량 → 폰 (양방향)

아두이노

```
// 아두이노: 200ms마다 상태를 폰으로
if (now - lastTx >= 200) {
  lastTx = now;
  BT.print(dist);  BT.print(',');
  BT.println(mode == AUTO ? "a" : "m");
}
```

- 앱의 Clock이 BytesAvailableToReceive를 확인하고, 있으면 ReceiveText로 한 줄을 읽어 Label에 표시합니다.
- 받은 a/m 값으로 Label_mode를 갱신하면 앱 표시와 차의 실제 모드가 일치합니다.

B 1인칭 시점(카메라 화면) 조종

- 3팀 STEP 1처럼 ESP32-CAM을 차에 달고, 차를 직접 보지 않고 **브라우저의 영상만 보며** 앱으로 조종합니다.
- 조종 코드는 바뀌지 않습니다. 달라지는 것은 조종하는 사람이 무엇을 보고 있느냐입니다.
- 처음에는 저속, 넓은 공간에서 연습합니다. 화면만 보면 거리감이 잘 안 잡힙니다.
- 영상에도 지연이 있어서 차가 이미 움직인 뒤에 화면이 따라옵니다. 그 지연이 몇 ms인지 재 보는 것도 좋은 과제입니다.
- ESP32-CAM과 PC는 팀에 배정된 Wi-Fi를 씁니다(3팀 Wi-Fi 배정표).

(드론에서 쓰는 FPV와 같은 개념이지만, 이 프로젝트에는 드론이 없고 RC카에 카메라를 단 것입니다.)

다음 단계: 이 코드를 4팀이 이렇게 씁니다

이어 갈 팀	2팀 코드가 쓰이는 곳
4팀	차량 쪽 블루투스 수신 코드를 그대로 쓰되, 보내는 쪽이 폰 앱이 아니라 PC 파이썬이 된다. 명령은 'g'(주행)와 's'(정지) 두 가지만 쓴다.

4팀 STEP 1 PC에서 블루투스 연결

4팀 STEP 3 차량 수신

2팀이 먼저 끝났다면 STEP 6의 '끊기면 멈춘다' 방식이 4팀의 핵심 아이디어와 같습니다. PC 파이썬에서 'g'나 's'를 보내는 짧은 프로그램을 만들어 차를 움직여 보세요(4팀 STEP 2, 3 참고).

3

3팀

영상 수신 + 색상 인식

차에 달린 카메라 영상을 무선으로 PC에 받아서, 빨간 표지판 같은 대상을 컴퓨터가 찾아 화면에 표시하는 프로그램을 만듭니다. 교재에 없는 파이썬을 새로 배웁니다. 4팀이 이 인식 코드를 가져다 씁니다.

먼저 되어 있어야 하는 것

교재에 없는 새 영역 (파이썬, OpenCV)

공통 · 전원과 안전

차례

01 시작 전: 새로 배우는 것

02 파이썬 기초 1~3

03 영상이 PC까지 오는 길

04 Wi-Fi와 접속 배정

05 STEP 1 ESP32-CAM 준비와 영상 확인

06 STEP 2 파이썬과 OpenCV 설치

07 STEP 3 영상 띄우기

08 HSV 색공간

09 STEP 4 색 범위로 걸러내기

10 STEP 5 슬라이더로 값 찾기

11 STEP 6 노이즈 정리

12 STEP 7 컨투어와 면적

13 STEP 8~9 대시보드, 측정

14 문제 해결과 완료 기준

15 다음 단계

3팀 시작 전

교재에 없는 새 영역: 파이썬과 영상

3팀은 교재 범위 밖의 파이썬과 OpenCV를 씁니다. 아두이노로 익힌 '센서 값을 읽고 기준과 비교해서 판단한다'는 흐름은 그대로 쓰이고, 언어와 도구가 바뀝니다.

준비 순서

1. 파이썬 기초 3장을 읽습니다 (다음 슬라이드).
2. 파이썬과 OpenCV를 설치합니다 (STEP 2).
3. ESP32-CAM에 예제를 올리고 브라우저로 영상을 봅니다 (STEP 1).
4. 파이썬으로 영상을 띄웁니다 (STEP 3).

그대로 쓰이는 것

- 센서 값, 기준값, 판단의 흐름
- 임계값을 직접 재서 정하는 습관
- 연속으로 확인해서 떨림을 막는 생각(4팀에서 사용)
- 시리얼 통신의 개념(4팀에서 사용)

새로 배우는 것

- 파이썬 문법 (들여쓰기, 콜론)
- 이미지를 숫자 배열로 다루는 것
- HSV 색공간, 마스크, 컨투어

파이썬 기초 1: 아두이노와 비교

하는 일	아두이노 (C++)	파이썬
변수	`int n = 5;`	`n = 5` (자료형을 쓰지 않음)
출력	`Serial.println(n);`	`print(n)`
조건	`if (a > b) { ... }`	`if a > b:` (콜론, 아랫줄 들여쓰기)
반복	`while (true) { ... }` `for (int i = 0; i < 5; i++)`	`while True:` `for i in range(5):`
함수	`int add(int a, int b) { return a + b; }`	`def add(a, b):` `  return a + b`
논리 연산, 주석	`&&  ||  !  // 설명`	`and  or  not  # 설명`
프로그램 구조	`setup()`과 `loop()`	위에서 아래로 실행, 반복은 `while True`

자주 틀리는 것 들여쓰기(공백 4칸), 줄 끝의 콜론(:), True와 False의 대소문자. 오류 메시지의 줄 번호부터 확인합니다.

파이썬 기초 2: 자주 쓰는 문법

연습용 코드 python 파일로 저장해서 실행

```python
name = "RC카"
speed = 150
print(f"{name} 속도 {speed}")        # 문자열 안에 값을 넣는다 (f-string)

lower = (0, 120, 70)                 # 튜플: 값 묶음 (HSV 범위에서 쓴다)
colors = ["red", "green", "blue"]    # 리스트
for c in colors:
    print(c)

def clamp(v, lo, hi):                # 함수: 값을 범위 안으로 자른다
    return max(lo, min(v, hi))

print(clamp(300, 0, 255))            # 255
print(max([], default=0))            # 비어 있으면 0 (4팀 코드에서 쓴다)
```

연습해 보기

- 위 코드를 실행해서 출력이 주석과 같은지 봅니다.
- clamp 함수로 -20, 100, 400을 넣어 봅니다.
- 리스트에 yellow를 추가하고 반복문으로 출력해 봅니다.
- 일부러 들여쓰기를 틀리게 해서 오류 메시지를 읽어 봅니다.

파이썬 기초 3: 이미지는 숫자 배열이다

- 카메라 영상 한 장(frame)은 **숫자 배열**입니다. 모양은 (높이, 폭, 3)이고, 3은 B, G, R 세 값입니다. 위 코드는 높이 360, 폭 640인 화면입니다.
- 배열은 **행(y), 열(x)** 순서로 접근하지만, OpenCV 그리기 함수는 **(x, y)** 순서를 씁니다. 이 차이가 자주 헷갈립니다.
- 특정 영역을 자르거나 색을 칠하는 일이 배열 계산 한 줄로 됩니다.
- 컬러는 BGR 순서입니다. 빨강은 (0, 0, 255)입니다.

연습용 코드

```python
import numpy as np

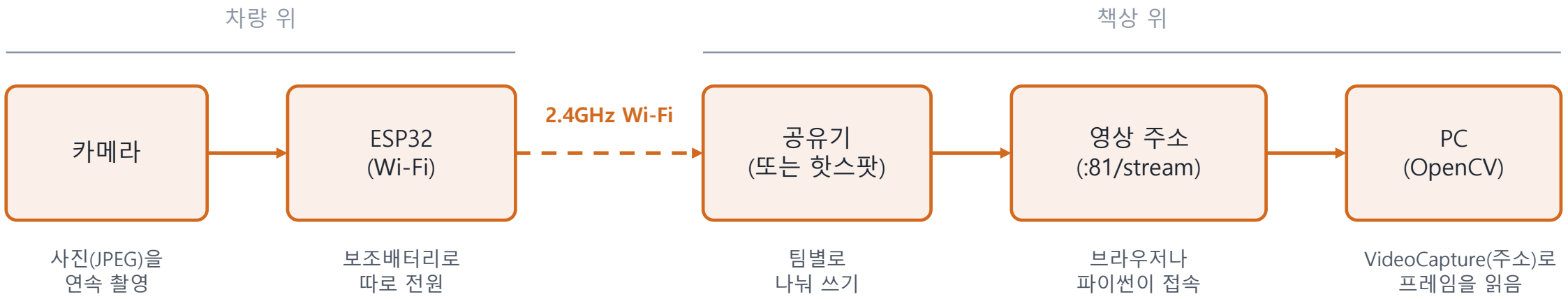
img = np.zeros((360, 640, 3), np.uint8)   # 검은 화면
img[100:200, 300:400] = (0, 0, 255)       # 빨강(BGR)으로 칠하기

print(img.shape)                           # (360, 640, 3)
print(img[150, 350])                       # [0 0 255]
```

확인 마지막 두 줄의 출력을 직접 눈으로 확인합니다. 영상 한 장이 이런 배열이라는 감각이 STEP 4~7의 바탕입니다.

영상이 PC까지 오는 길

ESP32-CAM은 카메라로 찍은 사진(JPEG)을 연속으로 Wi-Fi에 실어 보내고, PC는 웹 주소로 그 영상을 받습니다.



- 디지털 영상이라 줄무늬 노이즈는 없지만, Wi-Fi가 혼잡하면 화면이 멈칫하거나 늦어집니다.
- PC는 장치 번호 대신 영상 주소로 영상을 엽니다. 중간에 무선 구간이 있어 지연이 생길 수 있습니다(STEP 9에서 측정).
- ESP32-CAM은 모터와 **다른 전원(보조배터리)**을 씁니다. 전원이 흔들리면 영상이 끊기거나 보드가 재부팅됩니다.

3팀 원리

Wi-Fi와 접속: 네 팀이 동시에 켜면

- ESP32-CAM은 **2.4GHz Wi-Fi**에만 접속됩니다. 5GHz 전용이나 로그인 화면이 뜨는 학교 Wi-Fi에는 붙지 않습니다.
- 공유기 하나에 네 팀이 다 붙으면 영상이 느려질 수 있습니다. 가능하면 **팀별 Wi-Fi**(공유기 또는 휴대폰 핫스팟)를 씁니다.
- PC와 ESP32-CAM은 **같은 Wi-Fi**에 연결해야 영상 주소가 열립니다.
- 해상도는 QVGA(320×240)로 둡니다. 해상도를 올리면 끊김과 지연이 커집니다.
- 블루투스(2·4팀)도 같은 2.4GHz 대역을 쓰므로, 영상과 블루투스를 함께 켠 상태로 미리 시험합니다.
- ESP32-CAM은 오래 켜 두면 뜨거워집니다. 쉬는 시간에는 전원을 뺍니다.

Wi-Fi 배정표 (수업 시작 전에 칠판에 적기)

팀	Wi-Fi 이름	ESP32-CAM IP	확인
1팀			
2팀			
3팀			
4팀			

참고 IP 주소는 공유기가 정해 주므로 전원을 다시 켜면 바뀔 수 있습니다. 켤 때마다 시리얼 모니터에서 확인합니다.

ESP32-CAM 준비와 영상 확인

- 아두이노 IDE의 기본 설정 → 추가 보드 관리자 URL에 아래 주소를 넣고, 보드 매니저에서 **esp32 (Espressif Systems)**를 설치합니다.
  https://espressif.github.io/arduino-esp32/package_esp32_index.json
- ESP32-CAM을 MB 보드에 꽂고 USB로 연결합니다. 도구 → 보드에서 **AI Thinker ESP32-CAM**을, 포트에서 MB 보드의 COM 포트를 고릅니다.
- 파일 → 예제 → ESP32 → Camera → CameraWebServer를 엽니다. 카메라 모델을 **CAMERA_MODEL_AI_THINKER**로 고르고(예제 위쪽 또는 board_config.h), ssid와 password에 팀 Wi-Fi를 적습니다.
- 업로드한 뒤 시리얼 모니터를 **115200**으로 열고 MB 보드의 RST 버튼을 누릅니다. Camera Ready! 줄에 나온 IP 주소를 팀 노트에 적습니다.
- 같은 Wi-Fi에 연결한 PC 브라우저에서 IP 주소를 열고 **Start Stream**을 누릅니다. Resolution이 QVGA(320×240)인지 확인합니다.

확인

브라우저에 영상이 나옵니다. 업로드가 안 되면 MB 보드의 IO0 버튼을 누른 채 업로드를 시작하고, 영상이 끊기거나 재부팅을 반복하면 전원(보조배터리, USB 케이블)부터 확인합니다.

순서 브라우저에서 먼저 확인하면, 이후 파이썬에서 영상이 안 나올 때 '장비 문제'인지 '코드 문제'인지 구분할 수 있습니다. 확인이 끝나면 브라우저 탭을 닫습니다.

파이썬과 OpenCV 설치

- 파이썬(python.org)을 설치합니다. 설치 첫 화면의 **Add Python to PATH**를 체크합니다.
- VS Code 같은 편집기를 쓰면 편합니다(선택).
- 터미널(명령 프롬프트)에서 아래 명령을 실행합니다.
- 설치가 끝나면 마지막 명령으로 버전이 나오는지 확인합니다.

터미널에서 (한 줄씩)

```
pip install opencv-python numpy
pip install pyserial          # 4팀에서 사용

python -c "import cv2; print(cv2.__version__)"
```

pip를 못 찾는다고 나오면 py -m pip install opencv-python numpy 처럼 앞에 py -m을 붙여 보세요.

확인: 마지막 명령에서 4로 시작하는 버전 번호가 출력됩니다.

파이썬으로 영상 띄우기

먼저 되어 있어야 하는 것 STEP 1 STEP 2

코드 3-1

```python
import cv2, time

URL = "http://192.168.0.10:81/stream"   # STEP 1에서 적은 IP로 바꾼다

cap = cv2.VideoCapture(URL)
if not cap.isOpened():
    raise SystemExit("영상 주소에 연결하지 못했습니다")
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)     # 밀린 프레임 줄이기(지원될 때)

t_prev = time.perf_counter()
while True:
    ok, frame = cap.read()
    if not ok: break
    t_now = time.perf_counter()
    fps = 1.0 / (t_now - t_prev);  t_prev = t_now
    cv2.putText(frame, f"FPS {fps:.0f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)
    cv2.imshow("live", frame)
    if cv2.waitKey(1) == 27: break                # ESC로 종료
cap.release();  cv2.destroyAllWindows()
```

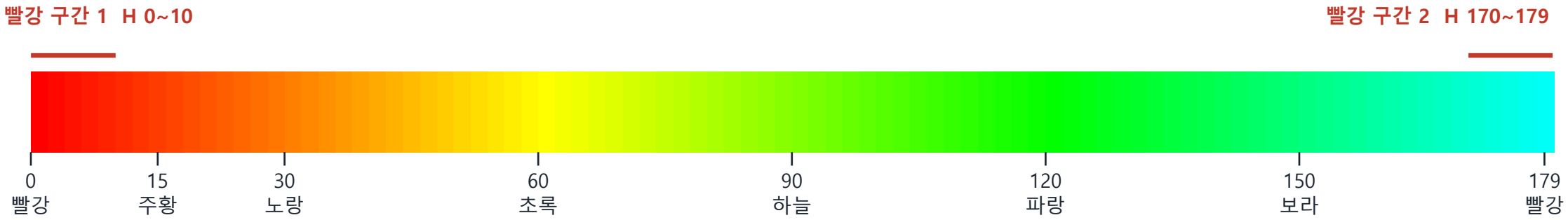
- 영상 주소는 STEP 1에서 적은 IP 뒤에 :81/stream을 붙인 것입니다. 브라우저에서 이 주소가 먼저 열리는지 확인합니다.
- 브라우저에서 영상을 보고 있으면 파이썬에서 안 열릴 수 있습니다(한 번에 한 곳에만 영상을 보냅니다). 탭을 닫습니다.
- 창에서 ESC를 누르면 종료합니다.

확인

창에 영상과 FPS가 나옵니다.

색을 숫자로: BGR 대신 HSV를 쓰는 이유

같은 빨강도 밝기가 바뀌면 B, G, R 세 값이 모두 변합니다. HSV는 '무슨 색인가(H)'를 따로 떼어 두어서 조명이 바뀌어도 H는 비교적 덜 변합니다.





S (채도) 0 = 흰색/회색에 가까움 → 255 = 선명한 색



V (명도) 0 = 어두움 → 255 = 밝음

OpenCV에서는 H가 0~179, S와 V가 0~255입니다. (일반 색상환의 0~360도를 반으로 나눈 값)

색 범위로 걸러내기 (inRange)

먼저 되어 있어야 하는 것 **STEP 3**

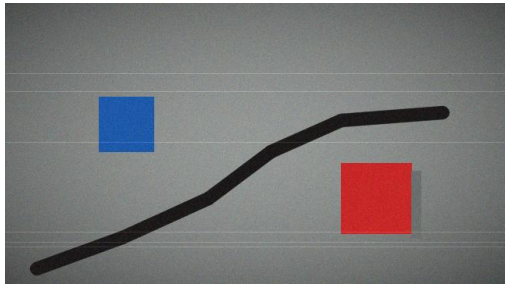
코드 3-2

```python
import numpy as np

lower1, upper1 = np.array([0, 120, 70]),   np.array([10, 255, 255])
lower2, upper2 = np.array([170, 120, 70]), np.array([179, 255, 255])

# 영상 읽기 루프 안에서
hsv  = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)       # BGR → HSV
mask = cv2.inRange(hsv, lower1, upper1) | cv2.inRange(hsv, lower2, upper2)
cv2.imshow("mask", mask)
```

- inRange는 범위 안의 픽셀을 흰색(255), 나머지를 검정(0)으로 만든 **마스크**를 돌려줍니다.
- 빨강은 H가 0 근처와 179 근처 두 곳에 있어서 두 범위를 합칩니다(|).
- OpenCV는 색을 BGR 순서로 다룹니다. RGB가 아닙니다.
- 빨강이 어려우면 파랑이나 초록 표지로 먼저 해 봐도 됩니다. 범위가 하나라서 쉽습니다.



원본



mask (빨간 상자만 흰색)

확인

빨간 표지판은 흰색으로, 바닥과 다른 물체는 검정으로 나옵니다. 아직 일부만 잡히거나 바닥이 같이 잡혀도 괜찮습니다. 다음 STEP에서 값을 찾습니다.

슬라이더로 값 찾기

먼저 되어 있어야 하는 것 STEP 4

코드 3-3

```python
def nothing(x): pass

cv2.namedWindow("tune")
cv2.createTrackbar("H_lo", "tune", 0,   179, nothing)
cv2.createTrackbar("H_hi", "tune", 10,  179, nothing)
cv2.createTrackbar("S_lo", "tune", 120, 255, nothing)
cv2.createTrackbar("V_lo", "tune", 70,  255, nothing)

# 매 프레임: 슬라이더 값을 읽어서 범위를 만든다
h_lo = cv2.getTrackbarPos("H_lo", "tune")
h_hi = cv2.getTrackbarPos("H_hi", "tune")
s_lo = cv2.getTrackbarPos("S_lo", "tune")
v_lo = cv2.getTrackbarPos("V_lo", "tune")
mask = cv2.inRange(hsv, (h_lo, s_lo, v_lo), (h_hi, 255, 255))
cv2.imshow("tune", mask)          # 마스크를 보면서 슬라이더를 움직인다
print(h_lo, h_hi, s_lo, v_lo)     # 만족스러운 값을 노트에 기록
```

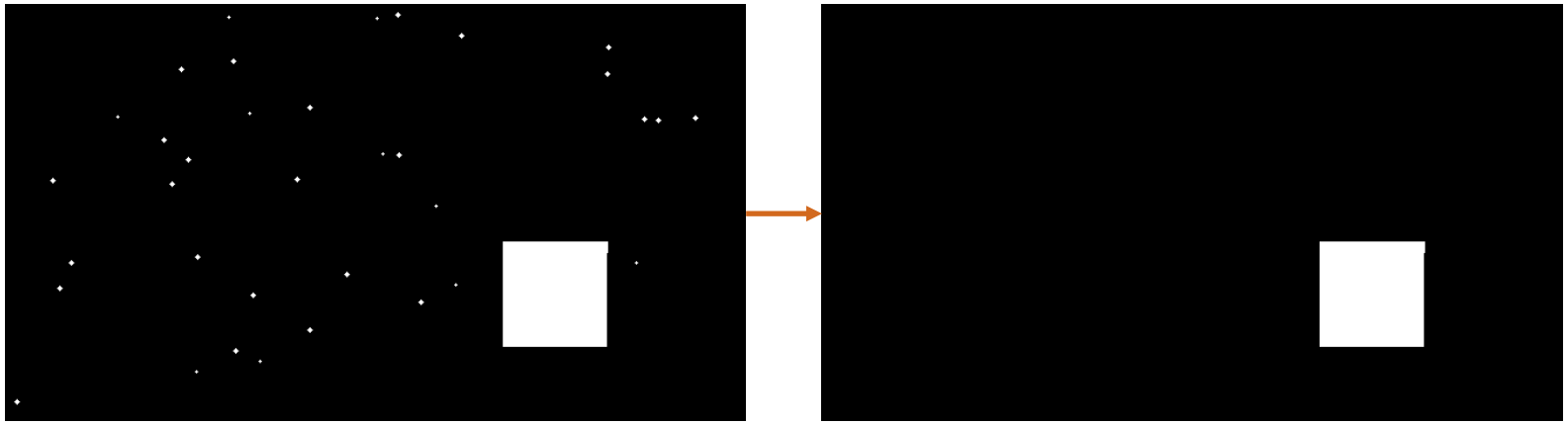
맞추는 순서

- S_lo, V_lo를 낮춰서 표지판이 하얗게 다 나오게 합니다.
- 바닥이나 그림자가 같이 켜지면 S_lo를 올립니다.
- 표지판 일부만 켜지면 V_lo를 낮춥니다.
- 빨강은 H 0~10 구간을 맞춘 뒤, 170~179 구간도 같은 방법으로 따로 맞춥니다.
- 조명이 바뀌면 다시 맞춥니다.

	H_lo	H_hi	S_lo	V_lo
교실 낮				
교실 저녁				

노이즈 정리: 열기와 닫기

먼저 되어 있어야 하는 것 STEP 5



정리 전: 작은 점이 남아 있다

정리 후

- **열기(OPEN)**: 깎은 뒤 다시 키웁니다. 작은 점이 사라집니다.
- **닫기(CLOSE)**: 키운 뒤 다시 깎습니다. 안쪽의 작은 구멍이 메워집니다.
- 커널이 클수록 강하게 지워지지만 작은 표지판도 같이 사라질 수 있습니다. 5×5에서 시작해 조정합니다.
- 다음 STEP에서 최소 면적(MIN_AREA)으로 한 번 더 걸러냅니다.

코드 3-4

```
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  kernel)   # 작은 점 지우기
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)   # 안쪽 구멍 메우기
```

확인: 표지판은 하나의 덩어리로, 바닥의 점은 사라집니다.

컨투어: 덩어리의 위치와 크기

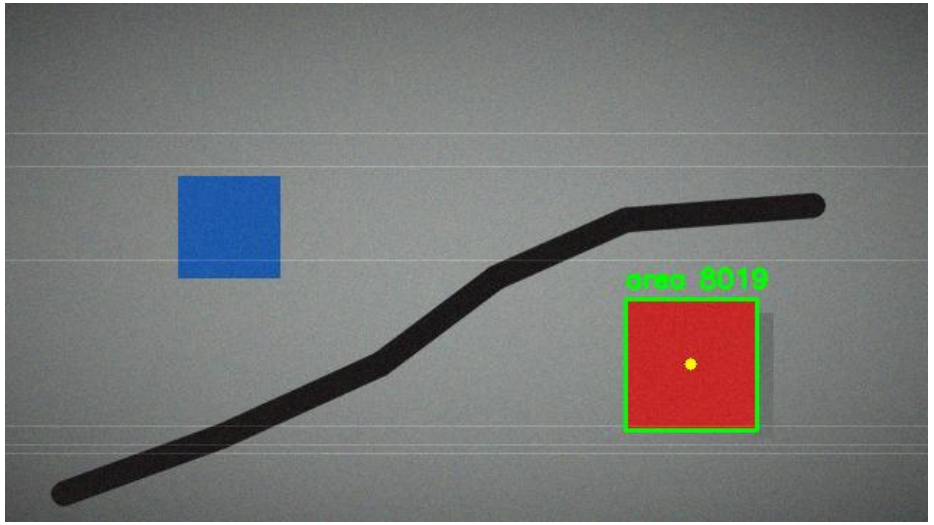
먼저 되어 있어야 하는 것 STEP 6

코드 3-5

```python
MIN_AREA = 500
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

best, best_area = None, 0
for c_ in cnts:
    a = cv2.contourArea(c_)
    if a > MIN_AREA and a > best_area:      # 가장 큰 덩어리를 고른다
        best, best_area = c_, a

if best is not None:
    x, y, w, h = cv2.boundingRect(best)
    cx, cy = x + w // 2, y + h // 2
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.circle(frame, (cx, cy), 4, (0, 255, 255), -1)
    cv2.putText(frame, f"area {int(best_area)}", (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```



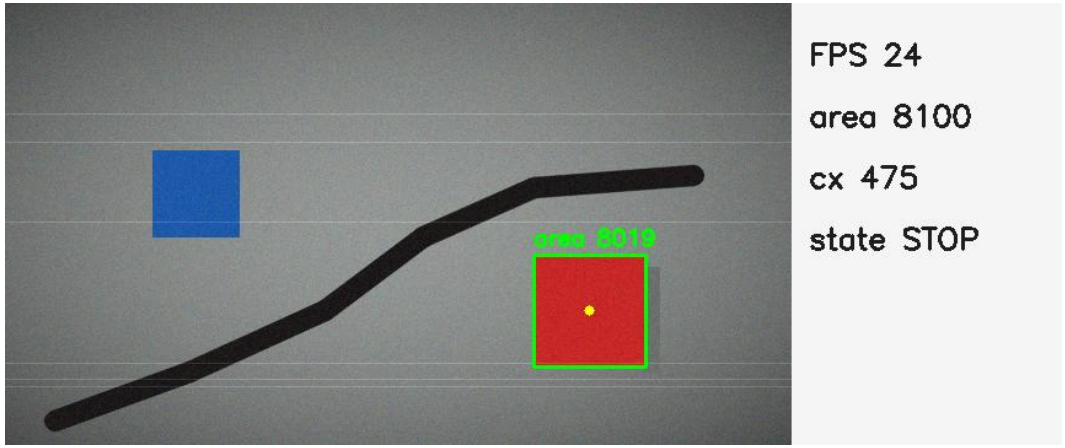
결과: 네모, 중심점, 면적이 표시된다

- **컨투어**는 마스크에서 흰 덩어리의 바깥 테두리입니다.
- 면적(area)은 표지판이 가까워질수록 커집니다. 4팀이 이 값을 보고 정지를 판단합니다.
- OpenCV 4는 값 2개를 돌려줍니다. 3.x 예제는 3개라서 'too many values' 오류가 날 수 있습니다.
- cv2.putText는 한글을 그리지 못합니다. 영어와 숫자로 씁니다.

확인: 표지판에 네모가 붙어 따라다니고 area 값이 바뀝니다.

대시보드: 한 화면에 모아 보기

먼저 되어 있어야 하는 것 STEP 7



예시: 영상 + 인식 결과 + 상태 패널

코드 3-6

```
panel = np.full((frame.shape[0], 220, 3), 245, np.uint8)
state = "STOP" if best_area > 3000 else "GO"      # 4팀 판단의 맛보기
info = [f"FPS {fps:.0f}", f"area {int(best_area)}",
        f"cx {cx if best is not None else '-'}", f"state {state}"]
for i, t in enumerate(info):
    cv2.putText(panel, t, (14, 50 + 50 * i),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (30, 30, 30), 2)
view = np.hstack([frame, panel])                  # 영상 옆에 패널을 붙인다
cv2.imshow("dashboard", view)
```

- 표시 항목: FPS, 면적, 중심 x좌표, 상태.
- 글자 크기와 배치는 발표 때 뒤에서도 읽히는지를 기준으로 정합니다.
- 여기서 쓴 3000은 임시 값입니다. 실제 값은 4팀 STEP 4에서 측정해서 정합니다.
- 대시보드 화면 구성은 4번째 팀원이 맡기 좋은 일입니다.

측정과 기록

'잘 된다'가 아니라 숫자로 말할 수 있어야 합니다. 거리와 조명을 바꿔 가며 검출이 몇 번 성공하는지 셉니다.

먼저 되어 있어야 하는 것 **STEP 7**

표지판 거리 ₩ 조명	밝음	보통	어두움
30cm			
60cm			
90cm			

칸에는 '검출 성공 횟수 / 시도 횟수'를 적습니다. 예: 18/20

분석 질문: 어느 조건에서 가장 많이 실패했나요? 그 원인은 무엇이라고 생각하나요?

FPS와 지연 보기

- STEP 3 화면의 FPS를 기록합니다.
- 카메라 앞에서 손을 흔들어 보고 화면이 얼마나 늦게 따라오는지 눈으로 확인합니다.
- 정확한 지연시간은 4팀 STEP 6의 스톱워치 촬영법으로 잽니다.

3팀

자주 막히는 곳과 완료 기준

증상	원인	조치
영상 주소가 안 열린다	IP가 바뀜, PC와 다른 Wi-Fi, 브라우저가 영상을 쓰는 중	시리얼 모니터로 IP 다시 확인, 같은 Wi-Fi 연결, 브라우저 탭 닫기
ESP32-CAM이 계속 재부팅된다	전원 부족	보조배터리로 따로 전원, 짧고 튼튼한 USB 케이블
영상이 끊기거나 늦다	Wi-Fi 혼잡, 해상도가 큼	해상도 QVGA, 팀별 Wi-Fi, 공유기 가까이에서 시험
빨강이 반만 잡힌다	빨강 구간이 둘인데 하나만 씀	두 범위를 | 로 합친다
바닥과 그림자도 잡힌다	S_lo, V_lo가 낮음	S_lo를 올린다
FPS가 낮다	처리가 무겁거나 해상도가 큼	imshow는 한 번만, 마스크 창은 필요할 때만
한글이 ?로 나온다	putText가 한글 미지원	영어·숫자로 표시

완료 기준

- ☐ 영상이 PC에 실시간으로 나온다
- ☐ 팀 Wi-Fi로 영상이 끊기지 않고 나온다
- ☐ 마스크에 표지판만 남는다
- ☐ 네모와 면적이 표시된다
- ☐ 거리·조명별 검출 기록표가 있다

다음 단계: 이 코드를 4팀이 이렇게 씁니다

이어 갈 팀	3팀 코드가 쓰이는 곳
4팀	STEP 4~7의 HSV 마스크, 정리, 컨투어, 면적 계산을 함수로 묶어서 그대로 사용합니다. 면적이 기준값을 넘으면 '정지'로 판단하고 블루투스로 차에 알립니다.
1팀 (심화)	영상 위에 차량 값을 글자로 표시하는 과제에서 STEP 3의 영상 보기 코드를 씁니다.
2팀 (심화)	1인칭 시점 조종에서 STEP 1의 브라우저 영상 보기를 씁니다.

4팀 STEP 4 판단 로직

4팀 STEP 5 통합 코드

3팀이 먼저 끝났다면 STEP 7의 중심점 cx를 이용해 '표지판이 화면 왼쪽에 있는가, 오른쪽에 있는가'를 구해 보세요. 검정 선을 같은 방법으로 찾으면 선이 화면 중앙에서 얼마나 벗어났는지(오차)를 영상으로도 구할 수 있습니다. 1팀 심화 A의 비례 제어와 연결되는 아이디어입니다.

4

4팀

영상 인식 → 정지 명령 → 주행

PC가 영상에서 빨간 표지판을 알아보면 블루투스로 차에 정지 명령을 보내고, 차가 멈추는 인지 → 판단 → 제어의 전체 흐름을 완성합니다. 1, 2, 3팀의 결과를 모두 씁니다.

먼저 되어 있어야 하는 것

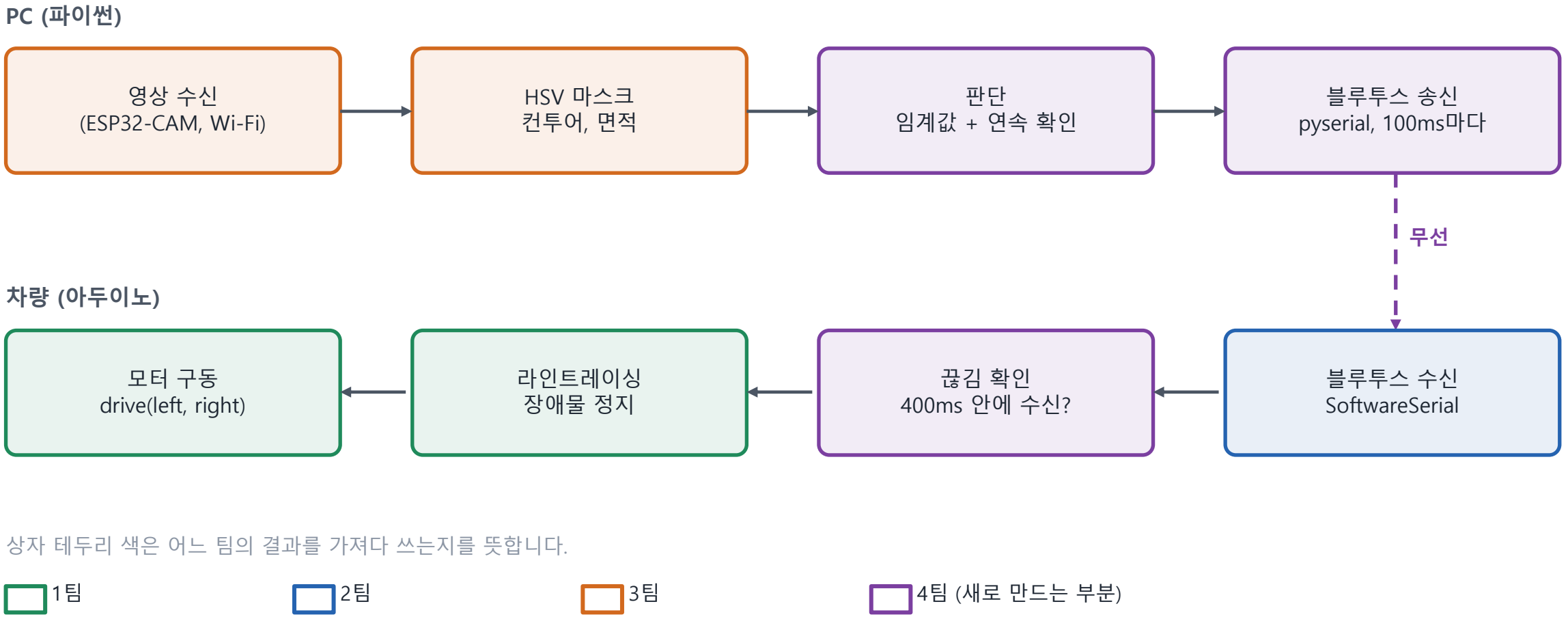
1팀 STEP 5, 6 라인트레이싱과 장애물 정지
2팀 STEP 1, 3 블루투스 수신
3팀 STEP 7 컨투어와 면적

차례

01 전체 흐름
02 시작 전 점검
03 STEP 1 PC에서 블루투스 연결
04 STEP 2 명령 보내 보기
05 STEP 3 차량 수신과 끊김 대비
06 STEP 4 판단 로직
07 STEP 5 통합 코드
08 STEP 6 지연시간 재기
09 STEP 7 시험과 기록
10 문제 해결과 완료 기준
11 확장 아이디어

한 장으로 보는 전체 흐름

새로 만드는 것은 '판단'과 'PC → 차량 송신'뿐입니다. 나머지는 앞 팀의 코드를 가져옵니다.



상자 테두리 색은 어느 팀의 결과를 가져다 쓰는지를 뜻합니다.

1팀 2팀 3팀 4팀 (새로 만드는 부분)

4팀

시작 전 점검

점검 항목	되어 있어야 하는 상태	안 되면 돌아갈 곳
라인트레이싱, 장애물 정지	선을 따라 달리고, 장애물 앞에서 멈춘다	1팀 STEP 5, STEP 6
블루투스 수신	터미널 앱으로 보낸 글자에 차가 반응한다	2팀 STEP 1, STEP 3
영상 수신과 인식	표지판에 네모와 area 값이 표시된다	3팀 STEP 3 ~ STEP 7

1팀 STEP 5 **1팀 STEP 6** **2팀 STEP 1** **2팀 STEP 3** **3팀 STEP 3** **3팀 STEP 7**

순서 세 가지가 따로따로 되는 것을 확인한 뒤에 합칩니다. 합친 다음 문제가 생기면 어느 부분이 원인인지 가려내기 어렵습니다.

PC에서 블루투스 연결하고 COM 포트 찾기

- 폰 앱 연결은 **끊어 둡니다**. 모듈은 한 번에 한 기기만 연결됩니다.
- 윈도우 설정 → Bluetooth 및 장치 → 장치 추가 → Bluetooth에서 HC-06을 찾아 페어링합니다(1234).
- 페어링하면 장치 관리자의 '포트(COM & LPT)'에 **Standard Serial over Bluetooth link (COMx)**가 생깁니다.
- 포트가 두 개 생기면 보통 하나는 수신(Incoming), 하나는 **발신(Outgoing)**입니다. 우리는 발신 포트를 씁니다. 설정의 '추가 Bluetooth 옵션 → COM 포트' 탭에서 방향을 확인합니다.
- 아래 명령으로 파이썬이 보는 포트 목록도 확인할 수 있습니다.

터미널에서 (pyserial 설치 후)

```
python -m serial.tools.list_ports
```

참고 연결 처음 몇 초는 시간이 걸립니다. 포트를 열고 2초쯤 기다린 뒤 보내는 것이 안전합니다.

확인: 발신 COM 포트 번호를 알아냈습니다. 번호를 팀 노트에 적습니다.

파이썬에서 명령 보내 보기

먼저 되어 있어야 하는 것 STEP 1

코드 4-1

```python
import serial, time

ser = serial.Serial("COM5", 9600, timeout=0.1)   # COM 번호는 본인 PC 기준
time.sleep(2)                                    # 연결 안정화
print("s: 정지,  g: 주행,  q: 종료")

while True:
    k = input("> ").strip().lower()
    if k == "q":
        break
    if k == "s": ser.write(b"s")
    if k == "g": ser.write(b"g")
ser.close()
```

- 차량은 2팀 STEP 3의 수신 코드(또는 STEP 3의 코드)를 올려 둡니다.
- s와 g를 입력해서 차가 반응하는지 확인합니다. (g는 2팀 앱의 '전진'과 달리 여기서는 '주행 허용'입니다.)
- PermissionError가 나오면 다른 프로그램이나 폰이 연결을 잡고 있는 것입니다.

확인: 키보드로 차를 멈췄다 출발시킬 수 있습니다.

차량 수신: 신호가 끊기면 멈춘다

먼저 되어 있어야 하는 것 **1팀 STEP 6** **2팀 STEP 3**

코드 4-2 차량 쪽 (아두이노)

```
SoftwareSerial BT(3, 4);   // RX=3, TX=4 (교재와 같음)
char cmd = 's';
unsigned long lastRx = 0;
const unsigned long LINK_MS = 400;

void loop() {
  while (BT.available()) {
    char c = BT.read();
    if (c == 'g' || c == 's') { cmd = c;  lastRx = millis(); }
  }
  bool linkOk = millis() - lastRx < LINK_MS;   // 최근 400ms 안에 신호가 있었나

  // dist, state(장애물 정지)는 1팀 STEP 6 코드 그대로
  if (state == STOPPED)             drive(0, 0);     // 장애물이 최우선
  else if (cmd == 'g' && linkOk)    lineTrace();     // 1팀 함수
  else                              drive(0, 0);     // 's'이거나 신호 끊김
}
```

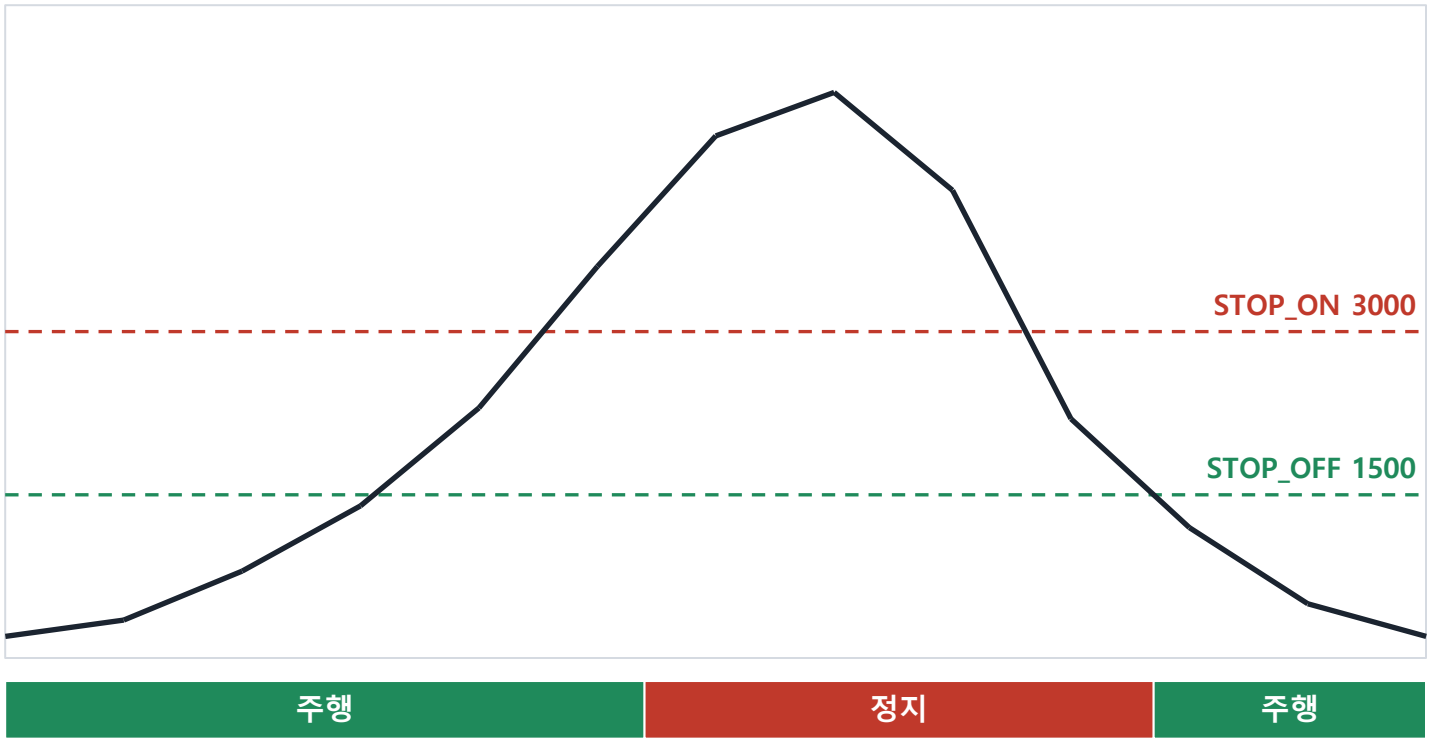
왜 'g'를 계속 보내나

- '정지' 한 번만 보내고 끝내면, 그 한 번이 유실되었을 때 차가 계속 달립니다.
- 그래서 PC가 100ms마다 **현재 상태**를 계속 보냅니다.
- 차는 400ms 안에 신호가 없으면 스스로 멈춥니다. PC 프로그램이 꺼지거나 블루투스가 끊겨도 안전합니다.
- 장애물 정지가 PC 명령보다 우선합니다.

판단 로직: 면적으로 정지를 결정한다

먼저 되어 있어야 하는 것 **3팀 STEP 7**

빨간 덩어리의 면적



- 면적이 STOP_ON(3000)을 넘으면 정지 후보입니다. 표지판이 가까울수록 면적이 커집니다.
- **연속 3프레임** 같은 판단이 이어져야 상태를 바꿉니다. 한 프레임의 노이즈는 무시합니다.
- 해제는 STOP_OFF(1500) 아래로 내려가야 합니다. 1팀의 15cm/20cm와 같은 생각입니다(히스테리시스).
- 3000과 1500은 임시 값입니다. 표지판을 정지하고 싶은 거리에 두고 area를 읽어서 정합니다.

통합 코드 (1) 준비와 인식 함수

3팀이 STEP 4~7에서 만든 인식 과정을 red_area() 함수 하나로 묶습니다. 이 함수는 영상 한 장을 받아 가장 큰 빨간 덩어리의 면적을 돌려줍니다.

먼저 되어 있어야 하는 것 **3팀 STEP 4~7** **STEP 2**

코드 4-3 준비

```python
import cv2, numpy as np, serial, time

ser = serial.Serial("COM5", 9600, timeout=0.05);  time.sleep(2)
cap = cv2.VideoCapture("http://192.168.0.10:81/stream")  # 3팀 STEP 1의 IP
K = np.ones((5, 5), np.uint8)
STOP_ON, STOP_OFF, CONFIRM = 3000, 1500, 3     # STEP 4에서 정한 값으로 바꾼다
state, cnt, last_tx = "GO", 0, 0.0

def red_area(frame):
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    m1 = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))
    m2 = cv2.inRange(hsv, (170, 120, 70), (179, 255, 255))
    m = cv2.morphologyEx(m1 | m2, cv2.MORPH_OPEN, K)
    m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, K)
    cnts, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    return max((cv2.contourArea(c) for c in cnts), default=0)
```

- 맨 위 네 줄: 블루투스 포트, 영상 주소, 정리용 커널, 기준값을 정합니다.
- red_area는 3팀 코드를 그대로 옮긴 것입니다. HSV 변환, 두 범위 합치기, 열기·닫기, 컨투어 면적 순서입니다.
- 덩어리가 하나도 없으면 default=0으로 면적 0을 돌려줍니다.
- 여기까지는 새로 만든 것이 없습니다.

통합 코드 (2) 판단하고 보내는 루프

먼저 되어 있어야 하는 것 **통합 코드 (1)** **STEP 4 판단**

코드 4-4 메인 루프

```
while True:
    ok, frame = cap.read()
    if not ok: break
    t0 = time.perf_counter()
    area = red_area(frame)

    stop_now = ((state == "GO" and area > STOP_ON) or
                (state == "STOP" and area >= STOP_OFF))
    want = "STOP" if stop_now else "GO"
    cnt = cnt + 1 if want != state else 0      # 바꾸자는 판단이 연속 몇 번째?
    if cnt >= CONFIRM:
        state, cnt = want, 0

    if time.time() - last_tx > 0.1:            # 100ms마다 현재 상태 전송
        ser.write(b"s" if state == "STOP" else b"g")
        last_tx = time.time()

    ms = (time.perf_counter() - t0) * 1000
    cv2.putText(frame, f"{state} area {int(area)} proc {ms:.0f}ms", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow("fullloop", frame)
    if cv2.waitKey(1) == 27: break

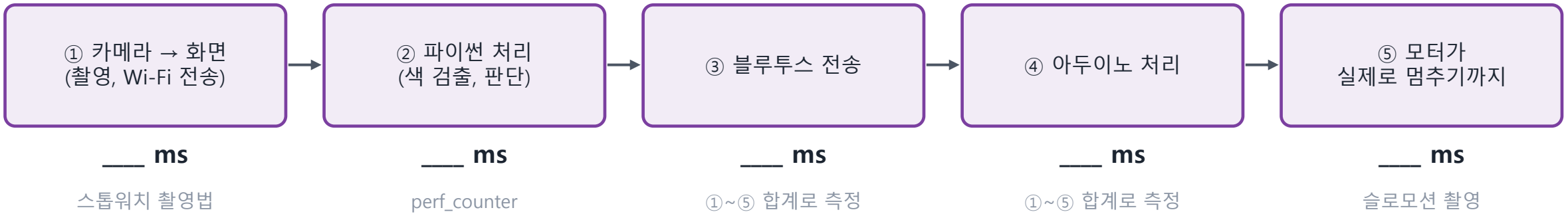
ser.write(b"s");  ser.close();  cap.release()
```

읽는 순서

- stop_now가 STOP_ON과 STOP_OFF 두 기준을 상태에 따라 구분합니다.
- cnt는 같은 판단이 연속으로 나온 횟수입니다. CONFIRM(3)번 이어져야 상태가 바뀝니다.
- 100ms마다 현재 상태를 보냅니다. 차는 이 신호가 끊기면 멈춥니다.
- 화면에 처리 시간(proc)을 표시해서 STEP 6 ②번 측정에 씁니다.
- 프로그램을 끝낼 때 마지막으로 s를 한 번 더 보냅니다.

지연시간 재기

표지판을 본 순간부터 바퀴가 멈추기까지는 여러 단계를 거칩니다. 단계마다 걸리는 시간을 나눠서 재 봅니다.



방법	하는 일
스톱워치 촬영법 (①)	밀리초가 보이는 스톱워치를 카메라 앞에 두고, 실제 스톱워치와 PC 화면 속 스톱워치가 한 장면에 같이 찍히게 사진을 찍습니다. 두 시각의 차이가 영상 지연입니다.
perf_counter (②)	코드의 proc 값을 읽습니다. 프레임 한 장을 처리하는 데 걸린 시간입니다.
슬로모션 촬영 (⑤)	표지판이 시야에 들어오는 순간부터 바퀴가 멈추는 순간까지를 스마트폰 슬로모션으로 찍고 프레임 수로 환산합니다. ③, ④는 이 값에 포함됩니다.

멈추는 거리 = 속도 × 전체 지연

예) 0.4m/s × 0.8초 = 32cm
속도는 2m 구간을 달리는 시간을 재서 구합니다.

시험과 기록

조건을 바꿔 가며 같은 시험을 반복합니다. 정지 후 표지판까지 남은 거리가 합격 기준(예: 5cm 이상) 안에 드는지 봅니다.

BASE 속도	조명	1회 (cm)	2회 (cm)	3회 (cm)	성공?
100	보통				
100	어두움				
130	보통				
130	어두움				
160	보통				
160	어두움				

합격 기준은 팀이 정해서 노트에 먼저 적고 시험합니다.

분석 질문

- 속도가 올라갈 때 멈추는 거리는 어떻게 변했나요?
- 어두울 때 면적 값은 어떻게 달라졌나요?
- 가장 개선 효과가 큰 방법은 무엇이었나요? (속도, STOP_ON, 처리 시간)

4팀

자주 막히는 곳과 완료 기준

증상	원인	조치
COM 포트를 열 수 없다	폰이나 다른 프로그램이 연결 중	폰 앱 끊기, 프로그램 종료
포트는 열리는데 차가 반응 없다	발신 포트가 아님, 속도 불일치	Outgoing 포트로 변경, 9600 확인
정지는 되는데 출발이 안 된다	STOP_OFF가 너무 낮거나 표지판이 시야에 남음	STOP_OFF 조정, 표지판 위치 확인
정지가 늦다	지연 + 속도	속도 낮추기, STOP_ON 낮추기, 처리 가볍게
정지와 출발을 반복한다	연속 확인과 히스테리시스 부족	CONFIRM을 늘리고 두 기준 간격을 넓힌다
차가 갑자기 멈춘다	400ms 끊김 대비 동작	블루투스 연결과 PC 처리 지연 확인
영상이 끊기거나 점점 늦어진다	Wi-Fi 혼잡, 밀린 프레임	해상도 QVGA, 팀별 Wi-Fi, BUFFERSIZE 1 설정(3팀 STEP 3)

완료 기준

- ☐ 키보드로 보낸 g/s에 차가 반응한다
- ☐ 표지판을 보면 스스로 멈춘다
- ☐ 치우면 다시 출발한다
- ☐ 블루투스를 끊으면 멈춘다
- ☐ 단계별 지연시간을 기록했다

4팀

더 해 볼 수 있는 것

- **신호등**: 빨강이면 정지, 초록이면 출발. 색 범위 두 개를 같이 다룹니다.
- **서행 구간**: 노랑을 보면 BASE 속도를 낮춰서 주행합니다. 명령을 'g'와 's' 외에 새 글자(예: 'v', 서행)로 늘립니다.
- **정지 후 대기**: 정지하고 N초 뒤 자동으로 출발합니다.
- **거리 추정**: 면적과 실제 거리의 관계를 측정해서 곡선으로 만들고, 거리 기준으로 정지를 판단합니다.
- **영상으로 조향**: 검정 선의 중심이 화면 중앙에서 얼마나 벗어났는지(오차)를 구해서 비례 제어로 조향합니다. 1팀 심화 A와 3팀 마지막 장이 출발점입니다.

순서 확장을 하기 전에 기본 흐름(인식 → 판단 → 전송 → 정지)이 반복해서 안정적으로 되는지 먼저 확인합니다.

부록

부록

용어 사전, 증상별 찾아가기, 추천 링크, 운영 규칙, 기록 양식을 모았습니다.

차례

01 용어 사전

02 증상별로 찾아가기

03 추천 링크

04 운영 규칙 (시작 전, 끝난 뒤)

05 기록 양식

부록

용어 사전

PWM	핀을 빠르게 켰다 껐다 해서 평균 전력을 조절하는 방법
듀티(duty)	한 주기에서 켜져 있는 시간의 비율
임계값	'이 값을 넘으면 ~이다'라고 판단하는 경계값
히스테리시스	켜는 기준과 끄는 기준을 다르게 둬서 떨림을 막는 방법
TOF	신호가 갔다 오는 데 걸린 시간으로 거리를 재는 방식
UART, 시리얼	글자(바이트)를 한 줄로 차례로 주고받는 통신 방식
보드레이트	시리얼 통신 속도(예: 9600). 양쪽이 같아야 함
페어링	블루투스 두 기기가 서로를 기억하고 연결하는 과정
상태머신	'지금 상태'를 변수로 기억하고 상태에 따라 동작을 바꾸는 구조
HSV	색을 색상(H), 채도(S), 명도(V)로 나눠 표현하는 방식
마스크	조건에 맞는 픽셀만 흰색으로 표시한 흑백 영상
컨투어	마스크 안 흰 덩어리의 바깥 테두리
지연시간(latency)	어떤 일이 일어난 뒤 결과가 나타나기까지 걸리는 시간
하트비트	연결이 살아 있음을 알리려고 주기적으로 보내는 신호
fail-safe	고장이나 끊김이 생겼을 때 안전한 쪽(정지)으로 가게 만든 설계

부록

증상별로 찾아가기

바퀴가 안 돌거나 한쪽만 돈다	**공통 · 값 확인 습관** **1팀 STEP 4**
선을 놓치거나 흔들린다	**1팀 STEP 2** **1팀 문제 해결**
장애물 앞에서 떤다	**1팀 STEP 6 기준 두 개**
블루투스 연결이 안 된다	**2팀 배선** **2팀 문제 해결**
폰이 아이폰이다	**2팀 원리**
영상이 안 나오거나 끊긴다	**3팀 Wi-Fi** **3팀 STEP 1** **3팀 문제 해결**
색이 잘 안 잡힌다	**3팀 HSV** **3팀 STEP 5**
PC에서 차에 명령이 안 간다	**4팀 STEP 1** **4팀 문제 해결**
정지가 늦다	**4팀 STEP 6 지연시간**

부록

추천 링크

공식 문서 중심으로 골랐습니다. 영어 문서가 많지만 코드 예제를 함께 보면 읽을 만합니다.

자료	주소	어디에 쓰나
Arduino 언어 레퍼런스	https://docs.arduino.cc/language-reference	함수 하나하나의 사용법
analogWrite() 설명	https://www.arduino.cc/reference/en/language/functions/analog-io/analogWrite	PWM 핀과 값의 범위
SoftwareSerial 예제	https://docs.arduino.cc/tutorials/communication/SoftwareSerialExample/	2팀 블루투스 배선·코드의 바탕
OpenCV: Changing Colorspaces	https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html	3팀 HSV와 inRange
MIT App Inventor 컴포넌트(연결)	https://ai2.appinventor.mit.edu/reference/components/connectivity.html	BluetoothClient 블록 설명
MIT App Inventor 시작	https://appinventor.mit.edu/	앱 만들기 시작 페이지
파이썬 공식 문서(한국어)	https://docs.python.org/ko/3/	3팀 파이썬 기초, 더 공부할 때
pySerial 짧은 소개	https://pyserial.readthedocs.io/en/latest/shortintro.html	4팀 포트 열기, 쓰기, 포트 목록
ESP32 보드 설치 (Espressif)	https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html	3팀 ESP32 보드 설치

부록

운영 규칙 : 시작 전과 끝난 뒤

시작 전

- ☐ 배터리 충전 상태 확인 (니켈수소 6개, 카메라용 보조배터리)
- ☐ Wi-Fi 배정표에서 우리 팀 Wi-Fi와 IP 확인
- ☐ ESP32-CAM 전원을 켜고 브라우저로 영상 확인
- ☐ 트랙과 장애물을 설치하고 바닥의 걸림 확인
- ☐ 처음 주행은 바퀴를 띄우고 방향 확인

끝난 뒤

- ☐ ESP32-CAM 전원(보조배터리)을 뺀다
- ☐ 보조배터리는 충전해 두고, 카메라 리본 케이블이 들뜨지 않았는지 확인한다
- ☐ 니켈수소는 6개를 함께 충전한다
- ☐ 바뀐 핀 번호, 측정값, 설정값을 노트에 정리한다
- ☐ 장비는 팀 번호 순으로 정해진 자리에 보관한다

부록

기록 양식 (실험 노트)

실습할 때마다 한 줄씩 적습니다. 발표와 평가에서 '왜 이렇게 정했는지'를 보여 주는 근거가 됩니다.

날짜	장소 / 조명	바꾼 것	설정값(임계값, 속도 등)	결과	다음에 할 것